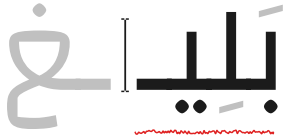
Cairo University
Faculty of Engineering
Department of Computer Engineering



Baligh

Arabic Writing Assistant

A Graduation Project Report Submitted

to

Faculty of Engineering, Cairo University

in Partial Fulfillment of the requirements of the degree

of

Bachelor of Science in Computer Engineering.

Presented by

Ahmed Hamed Gaber Hamed **Akram Hany Karam Salam**
Amir Anwar Bekhit Awd Kedis **Somia Saad Ismail El-Shemy**

Supervised by

Dr. Ayman AboElhassan

July, 2026

Abstract

Arabic digital writing still suffers from a shortage of integrated tools that can correct Modern Standard Arabic text accurately, respond in real time, and explain their feedback in a way that helps users learn. Existing tools often focus on surface spelling fixes or black-box suggestions without exposing the grammatical reasoning behind the correction. Baligh addresses this problem by providing an Arabic writing assistant that combines grammatical error detection, grammatical error correction, spelling-aware editing, automatic vowelization (Tashkeel), next-word suggestion, and explanatory feedback in a single user-facing system. The project aims to support students, educators, and Arabic content writers by improving writing quality while also making the correction process more understandable and educational.

The implemented system follows a modular pipeline. A preprocessing layer performs normalization, word-boundary handling, segmentation, and morphological analysis and disambiguation. On top of that, Baligh integrates a grammatical error detection engine that fuses rule-based, lexicon-based, and machine-learning detectors, then passes its findings to correction components built around ontology-guided reasoning, dictionary lookup, text-editing models, and automatic vowelization support. In parallel, a next-word suggestion module supports both next-word prediction and word auto-completion. The final system highlights issues, proposes corrections, returns ranked suggestions, generates Tashkeel when needed, and presents grammar-oriented explanations to the user in an interactive workflow.

The project outputs include the preprocessing pipeline, grammatical error detection and correction services, the next-word suggestion subsystem, and a working user application. Testing and validation relied on automated unit and integration tests together with evaluation on established Arabic linguistic resources, including QALB14, QALB15, and ZAEBUC. Overall, Baligh demonstrates that a modular and explainable Arabic writing assistant can combine linguistic processing and machine learning into a practical platform for improving Arabic writing quality.

الملخص

لا تزال الكتابة الرقمية باللغة العربية تعاني من نقص الأدوات المتكاملة التي تستطيع تصحيح نصوص العربية الفصحى بدقة، والعمل بزمن استجابة مناسب، وتقديم تفسير واضح يساعد المستخدم على التعلّم بدل الاكتفاء بإظهار التصحيح فقط. فكثير من الأدوات المتاحة تركز على الأخطاء الإملائية السطحية أو تقدم اقتراحات غير مفسرة. يعالج مشروع بليغ هذه المشكلة من خلال بناء مساعد كتابة عربي يجمع بين اكتشاف الأخطاء النحوية، وتصحيحها، ومعالجة الأخطاء الإملائية، واقتراح الكلمة التالية، وتقديم تفسير مبسط للقاعدة أو سبب الخطأ داخل نظام واحد موجه للمستخدم. ويهدف المشروع إلى خدمة الطلاب والمعلمين وصناع المحتوى العربي عبر تحسين جودة الكتابة وجعل عملية التصحيح أكثر فائدة من الناحية التعليمية.

يعتمد النظام المنفذ على بنية معيارية تبدأ بطبقة للمعالجة المسبقة تشمل التطبيع، وتحديد حدود الكلمات، والتجزئة، ثم التحليل الصرفي وفك الالتباس. بعد ذلك يدمج بليغ محركاً لاكتشاف الأخطاء النحوية يعتمد على المزج بين قواعد لغوية، وكشافات معجمية، ونماذج تعلم آلي، ثم يمرر النتائج إلى وحدات التصحيح المبنية على الاستدلال المعتمد على الأنطولوجيا، والبحث المعجمي، ونماذج التحرير النصي. وبالتوازي مع ذلك، يوفر النظام وحدةً لاقتراح الكلمة التالية تشمل التنبؤ بالكلمة التالية والإكمال التلقائي للكلمة الجارية كتابتها. وتعرض المنصة النهائية الأخطاء والتصحيحات والاقتراحات والتفسيرات اللغوية بصورة تفاعلية.

تشمل مخرجات المشروع خط معالجة مسبقة متكاملاً، وخدمات لاكتشاف الأخطاء وتصحيحها، ووحدة اقتراح الكلمات، وتطبيقاً عملياً للمستخدم. واعتمد التحقق من النظام على اختبارات آلية للوحدات والتكامل، إضافة إلى تقييم مبني على موارد عربية معروفة مثل QALB14 وQALB15 وZAEBUC. وتُظهر المحصلة أن بليغ يقدم نموذجاً عملياً لمساعد كتابة عربي قابل للتفسير يجمع بين المعالجة اللغوية والتعلم الآلي لخدمة الكتابة العربية بصورة أكثر جودة وفائدة.

Acknowledgment

Baligh was not built through technical effort alone, but through the support, patience, and trust of many people around us. We would first like to thank our supervisor, Dr. Ayman AboElhassan, for his steady guidance throughout this project. His questions, feedback, and encouragement pushed us to think more carefully, work more seriously, and keep improving the project whenever we felt uncertain or stuck.

We are also deeply grateful to the Faculty of Engineering at Cairo University and the Department of Computer Engineering. The years we spent there shaped the way we think, learn, and solve problems, and this project reflects much of what we gained from that journey. Baligh became possible because of the academic foundation and sense of responsibility that this environment gave us.

We would also like to thank one another as a team. This project demanded time, patience, repeated trial and error, and many long discussions before ideas became clear and features became real. What made that process meaningful was not only the final result, but the way we supported each other through the difficult parts, shared the pressure, and kept moving forward together.

Finally, we thank our families and loved ones with all sincerity. Their patience during busy and stressful periods, their constant encouragement, and their belief in us gave us the strength to continue. Any achievement in this work carries a part of their support within it, and for that we are truly grateful.

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
GEC	Grammatical Error Correction
GED	Grammatical Error Detection
MSA	Modern Standard Arabic
NLP	Natural Language Processing
NWP	Next-Word Prediction

Contacts

Team Members

Name	Email	Phone Number
Ahmed Hamed Gaber Hamed	ahmed.hamed03@eng-st.cu.edu.eg	01226968657
Akram Hany Karam Salam	akram.sallam03@eng-st.cu.edu.eg	01148746563
Amir Anwar Bekhit Awd Kedis	amir.awd03@eng-st.cu.edu.eg	01221461992
Somia Saad Ismail El-Shemy	somia.elshemy02@eng-st.cu.edu.eg	01080732462

Supervisor

Name	Email	Phone Number
Dr. Ayman AboElhassan	ayman.abo.elmaaty@eng.cu.edu.eg	01060836189

Chapter 1: Introduction

Baligh is an Arabic writing assistant designed to support Modern Standard Arabic writing through explainable correction and real-time assistance. This chapter introduces the motivation behind the project, defines the problem it addresses, summarizes the main outcomes delivered by the project, and outlines the organization of the remainder of the report.

1.1. Motivation and Justification

Arabic digital writing still lacks broadly available tools that combine grammatical support, writing speed, and educational value in a single user experience. While many writing assistants for other languages provide immediate correction, prediction, and revision support, Arabic users often rely on tools that address only isolated spelling mistakes, provide limited grammatical help, or return suggestions without clarifying the linguistic reason behind them. This gap is especially important in Modern Standard Arabic, where accurate writing is influenced by rich morphology, orthographic ambiguity, and context-sensitive grammatical structure.

These characteristics make Arabic NLP more demanding to model and more difficult to serve well in real time. A useful writing assistant must do more than detect surface-level errors; it must process text carefully, understand partially typed input, and return feedback quickly enough to support natural writing. It must also present corrections in a way that is understandable to users rather than functioning as a black box.

Baligh is motivated by this need for an integrated Arabic writing assistant that combines grammatical error detection, grammatical error correction, automatic vowelization (Tashkeel), next-word assistance, and explanatory feedback in one workflow. The project is intended to benefit students, educators, and Arabic content writers who need writing support that improves both text quality and user understanding at the same time.

1.2. Project Objectives and Problem Definition

The primary objective of Baligh is to build an explainable writing assistant for Modern Standard Arabic that can help users write more accurately and efficiently. To achieve this objective, the project integrates a preprocessing pipeline for normalization, seg-

mentation, and morphological analysis with a grammatical error detection subsystem, grammatical error correction components, an automatic vowelization (Tashkeel) capability, a next-word suggestion module, and a user-facing application interface. The system is designed to highlight issues, generate ranked suggestions, provide next-word and auto-completion support, generate vowelized output when needed, and present explanations that make the feedback more useful for learning.

In addition to functional assistance, the project aims to demonstrate that Arabic writing support can be delivered through a modular architecture that balances linguistic processing, machine learning, usability, and responsiveness. This makes Baligh not only a correction tool, but also a platform for combining multiple Arabic NLP capabilities into a practical workflow.

Formally, the problem addressed by this project is the lack of an integrated and explainable system for Modern Standard Arabic writing that can detect and correct grammatical issues, assist users during text entry, and deliver reliable feedback in a form suitable for real-time interactive use.

1.3. Project Outcomes

The project produced a working set of interconnected software components that together form the Baligh writing assistant. The implemented outcomes include a preprocessing pipeline responsible for text normalization, word-boundary handling, segmentation, and morphological analysis, as well as a grammatical error detection subsystem that combines rule-based, lexicon-based, and machine-learning detectors through a fusion layer.

The project also delivered grammatical error correction components that use ontology-based reasoning, dictionary-based lookup, and text-editing approaches to generate and rank correction candidates, together with automatic vowelization support for Arabic text. In parallel, Baligh includes a next-word suggestion subsystem that supports both next-word prediction and word auto-completion for interactive writing scenarios.

At the application level, the project provides a backend API and a web-based client that expose these capabilities to the user through an integrated writing workflow. Testing and evaluation were supported through automated checks and experiments on established Arabic resources, including QALB14, QALB15, and ZAEBUC, in addition to the accompanying project documents and technical materials.

1.4. Document Organization

The remainder of this report is organized as follows. Chapter 2 presents the Market Visibility Study, which examines the intended users of the system, reviews related products, and discusses the practical need for Baligh.

Chapter 3 provides the Literature Survey, covering the Arabic NLP background and the research context most relevant to the project. Chapter 4 describes the System Design and Architecture, including the main modules, their responsibilities, and the overall data flow.

Chapter 5 discusses System Testing and Verification, explaining how the implemented components were checked and evaluated. Finally, Chapter 6 presents the Conclusions and Future Work, summarizing the project contributions and outlining possible directions for extending Baligh in later stages.

Chapter 2: Market Visibility Study

This chapter surveys the market related to the project and discusses similar tools or platforms. It should explain their limitations and justify the need for Baligh.

2.1. Targeted Customers

Baligh is designed to assist individuals and organisations that frequently produce Arabic text and require high grammatical accuracy and proper vowelisation. The system targets both educational and professional sectors, where maintaining linguistic correctness is essential.

The primary target customers include:

- **Students and Arabic Language Learners**
  Students at schools and universities often struggle with Arabic grammar (Nahw) and the correct application of I'rab. Baligh provides real-time grammatical corrections and automatic vowelization.
- **Teachers and Educational Institutions**
  Arabic language teachers can use Baligh as an educational assistant to review assignments, prepare teaching materials, and provide immediate feedback to students.
- **Researchers and Academic Writers**
  Researchers who prepare scientific papers, theses, or publications in Arabic require grammatically correct and professionally written text.
- **Content Creators and Journalists**
  Writers, journalists, bloggers, and digital content creators regularly produce large volumes of Arabic text. Baligh accelerates the proofreading process while maintaining linguistic accuracy.
- **Government and Business Organizations**
  Government agencies and companies frequently produce official reports, legal documents, and correspondence in Arabic. Baligh reduces manual proofreading effort.

Customer Benefits:

- Real-time grammar correction based on Arabic Nahw and I’rab rules.
- Context-aware automatic vowelization (Tashkeel).
- Intelligent writing suggestions during typing.
- Reduced proofreading time.
- Improved readability and writing quality.
- Educational explanations that help users learn from their mistakes.

2.2. Market Survey

Although several Arabic writing assistants currently exist, none provides a comprehensive solution that combines grammatical reasoning, contextual vowelization, and real-time intelligent suggestions. Most existing systems specialize in only one aspect of Arabic language processing.

The following sections discuss the most relevant competing products.

2.2.1. Qalam AI

Qalam AI is an AI-powered Arabic writing assistant designed to improve spelling and grammar. It also supports automatic Tashkeel generation for Arabic text.

Strengths

- User-friendly interface.
- AI-assisted grammar and spelling correction.

Weaknesses

- Does not explain grammatical errors.
- Does not explicitly perform Nahw and I’rab analysis.
- Doesn’t perform text completion.
- Doesn’t correct diacritic mistakes.
- Not free.

Figure 2.1: *Wrong Correction*

Figure 2.2: *No Diacritiation Correction*

Figure 2.3: *Missed A Grammatical Mistake*

Figure 2.4: *Missed Many Grammatical Mistakes*

2.2.2. Sahehly

Sahehly is an Arabic proofreading platform that combines spelling correction, grammar checking, punctuation correction, and automatic Tashkeel.

Strengths

- Full-text diacritization.
- Doesn’t detect diacritics mistakes.

Weaknesses

- Limited real-time interaction while typing.
- No explicit grammatical reasoning or I’rab analysis.
- Premium features require a subscription.

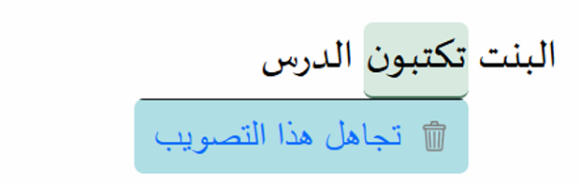


Figure 2.5: *Wrong Correction*

Figure 2.6: *No Diacritization Correction*

Figure 2.7: *Missed The grammatical Error*

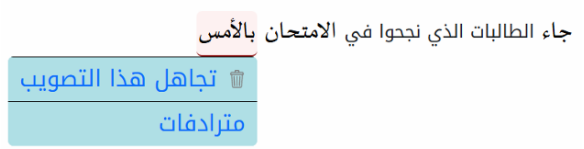


Figure 2.8: *Missed Many Grammatical Errors*

2.2.3. AlNahawi

Alnahawi is an AI-powered Arabic writing assistant designed to improve Arabic text through grammar correction and spelling correction.

Strengths

- Provides grammar and spelling correction.
- Simple and user-friendly interface.

Weaknesses

- Does not provide detailed grammatical explanations based on Nahw and I’rab.
- Doesn’t perform text completion.
- Not fully free.

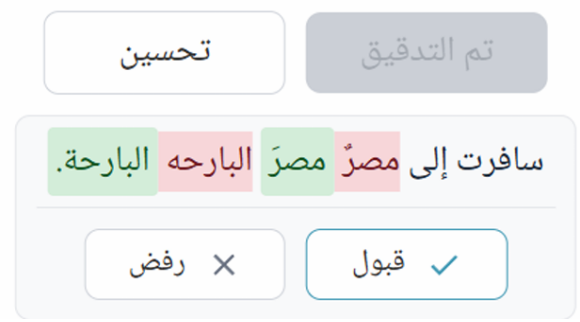


Figure 2.9: *AlNahawi Interface (1)*

Figure 2.10: *AlNahawi Interface (2)*

Figure 2.11: *AlNahawi Interface (3)*

Figure 2.12: *AlNahawi Interface (4)*

2.2.4. Arwi

Arwi is an AI-based Arabic writing platform.

Strengths

- Provides error detection.

Weaknesses

- Doesn’t perform error correction.
- Doesn’t perform text completion.
- Doesn’t correct diacritic mistakes.
- Does not provide detailed grammatical explanations based on Nahw and I’rab.
- Not free.

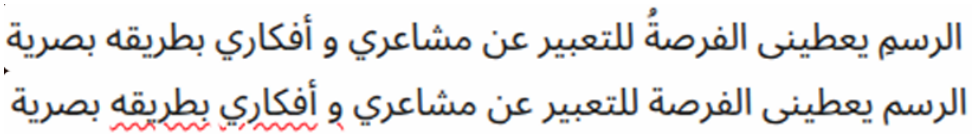


Figure 2.13: *Marked a correct word as a mistake and no Diacritics correction*

2.3. Business Case and Financial Analysis

2.3.1. Business Model

Baligh can be offered using a Software-as-a-Service (SaaS) model with multiple subscription plans targeting individual users, educational institutions, and enterprise customers.

Potential revenue streams include:

- Individual monthly subscriptions.
- Educational institution licenses.
- Enterprise API access.

2.3.2. Pricing Assumptions

Pricing strategy is shown below.

Table 2.1: *Pricing Strategy*

Plan	Monthly Price
Free	$0
Student	$5
Individual	$10
Enterprise	Custom quotation

2.3.3. Capital Expenditure (CapEx)

Estimated initial investment: **$0**

2.3.4. Operational Expenditure (OpEx)

Monthly operating expenses include: **$10** for **API Hosting**

2.3.5. Revenue Projection

Assuming:

- 2,000 active users.
- 15% subscribe to the Student plan.
- 5% subscribe to the Individual plan.

Estimated monthly revenue:

- Student subscriptions: $300 \times \$5 = \$1{,}500$
- Professional subscriptions: $100 \times \$10 = \$1{,}000$

Total monthly revenue: **$2,500**

Additional enterprise licensing and API subscriptions could significantly increase revenue.

2.3.6. Cash Flow

With a CapEx of $0 and an OpEx of only $10 per month, the project has a very low ongoing operating cost. As a result, the cash flow profile becomes highly favourable and even a small number of paying users would be sufficient to cover monthly expenses.

2.3.7. Break-Even Analysis

The system is expected to reach break-even almost immediately after launch, and any subscription revenue beyond the monthly operating cost would generate positive cash flow from the start.

Chapter 3: Literature Survey

This chapter summarizes the Arabic NLP background most relevant to Baligh and reviews the detection, correction and prediction approaches that influenced the system design.

3.1. Background on Arabic NLP

Arabic writing assistance is difficult because Arabic is morphologically rich, often undiacritized, and split across several language varieties [1, 2]. In Modern Standard Arabic, clitics attach to stems and words carry substantial grammatical information, so segmentation and morphological analysis are essential preprocessing steps. Tools such as Farasa and CAMeL Tools are therefore important foundations for downstream correction and prediction [1, 3].

Ambiguity is increased by the absence of short vowels in ordinary writing and by frequent orthographic issues such as Hamza variation, Ta-Marbuta confusion, punctuation mistakes, and merge or split errors [2, 4]. This means Arabic writing support must distinguish among orthographic, morphological, and syntactic errors rather than treat everything as spelling.

The main resources for Arabic GEC are still limited compared with English. QALB-2014 and QALB-2015 remain the primary benchmarks, while ZAEBUC and Tibyan extend coverage with additional corrected data and richer annotation [5–7]. Treebanks such as PADT, CAMeL Treebank, and I3rab are also useful because they expose grammatical structure that can support detection and explanation [8–10]. For next-word assistance, efficient language modeling remains important, with KenLM offering a practical low-latency option and neural models offering stronger context modeling at higher cost [11–13].

3.2. Background on Writing Assistance Systems

Writing assistants help users during text entry, not only after writing is complete. In Arabic, this usually requires a pipeline for normalization, segmentation, error detection, correction, ranking, and feedback presentation, with next-word assistance added when interactive typing support is needed.

Arabic GEC literature mainly falls into three families. Rule-based systems use hand-crafted rules, lexicons, and regular expressions, as in SAHSOH@QALB-2015, and are attractive because they are interpretable [5]. Ontology-based systems, such as the approach of Moukrim et al., encode grammar more explicitly and support explanation-oriented correction [14]. Neural systems improve coverage and benchmark performance, especially with Transformer-based or pretrained seq2seq models [15,16]. Their main trade-off is lower interpretability and higher deployment cost.

Text-editing GEC is particularly important for Baligh. Instead of freely generating a corrected sentence, it predicts token-level edits, which makes the output faster and easier to align with user-visible changes. Alhafni and Habash showed that this approach is both accurate and efficient for Arabic [17].

Corrected	النفسية ٠ *Alnfsyħ*		الصحة *AlSHħ*			سيما *symA*	ولا *wlA*	بالصحة *bAlSHħ*				الاهتمام *AlAhtmAm*			يجب *yjb*				(a)
	7		6		5	4		3			2	1			0					
Erroneous	النفسيه *Alnfsyh*		الصحه *AlSHh*		في *fy*	ولاسيما *wlAsymA*		لصحه *lSHh*			ب *b*	الإهتمام *AlĂhtmAm*			يجب *yjb*				(b)
Word Edits	KKKKKKR_[ة]A_[.]		KKKKR_[ة]		DD	KKKI_[ ]KKKK		MI_[ا]KKKR_[ة]			K	KKR_[ا]KKKKK			KKK				(c)
Word Edits (Compressed)	K*R_[ة]A_[.]		K*R_[ة]		D*	KKKI_[ ]K*		MI_[ا]K*R_[ة]			K*	KKR_[ا]K*			K*				(d)
	7b	7a	6b	6a	5	4		3b	3a		2	1b	1a		0					
Tokenized Erroneous	ه## *##h*	النفسي *Ālnfsy*	ه## *##h*	الصح *AlSH*	في *fy*	ولاسيما *wlAsymA*		ه## *##h*	لصح *lSH*		ب	هتمام## *##htmAm*	الإ *AlĂ*		يجب *yjb*				(e)
Subword Edits	R_[ة]A_[.]	KKKKKK	R_[ة]	KKKK	DD	KKKI_[ ]KKKK		R_[ة]	MI_[ا]KKK		K	KKKKK	KKR_[ا]		KKK				(f)
Subword Edits (Compressed)	R_[ة]A_[.]	K*	R_[ة]	K*	D*	KKKI_[ ]K*		R_[ة]	MI_[ا]K*		K*	K*	K*R_[ا]		K*				(g)

Figure 3.1: *Example of text-editing tags for Arabic grammatical error correction.*

Writing assistants also need grammatical error detection and useful feedback. Treating GED as a sequence labeling task can improve interpretability, and resources such as ARETA help classify correction outputs into meaningful error types [4, 16]. For next-word assistance, n-gram models remain attractive for low latency, while neural and character-aware models offer stronger contextual modeling when more computation is acceptable [11, 12, 18]. ARWI is a strong example of a more complete Arabic writing assistant built around these ideas [19].

3.3. Comparative Study of Previous Work

Previous work shows a clear trade-off. Symbolic systems are more explainable, but neural systems usually provide broader coverage and better benchmark results. Text-editing models offer a useful compromise because they keep the correction output structured while remaining much faster than fully generative systems [5, 14, 15, 17].

The literature also shows that strong Arabic writing support depends on more than one model. Preprocessing tools such as Farasa and CAMeL Tools are needed before correction, benchmark datasets shape what systems can learn, and predictive text models address a separate but complementary part of the user experience [1, 3, 11]. The main gap is therefore integration: many papers solve one subproblem well, but fewer systems combine correction, prediction, and explanation in one practical workflow.

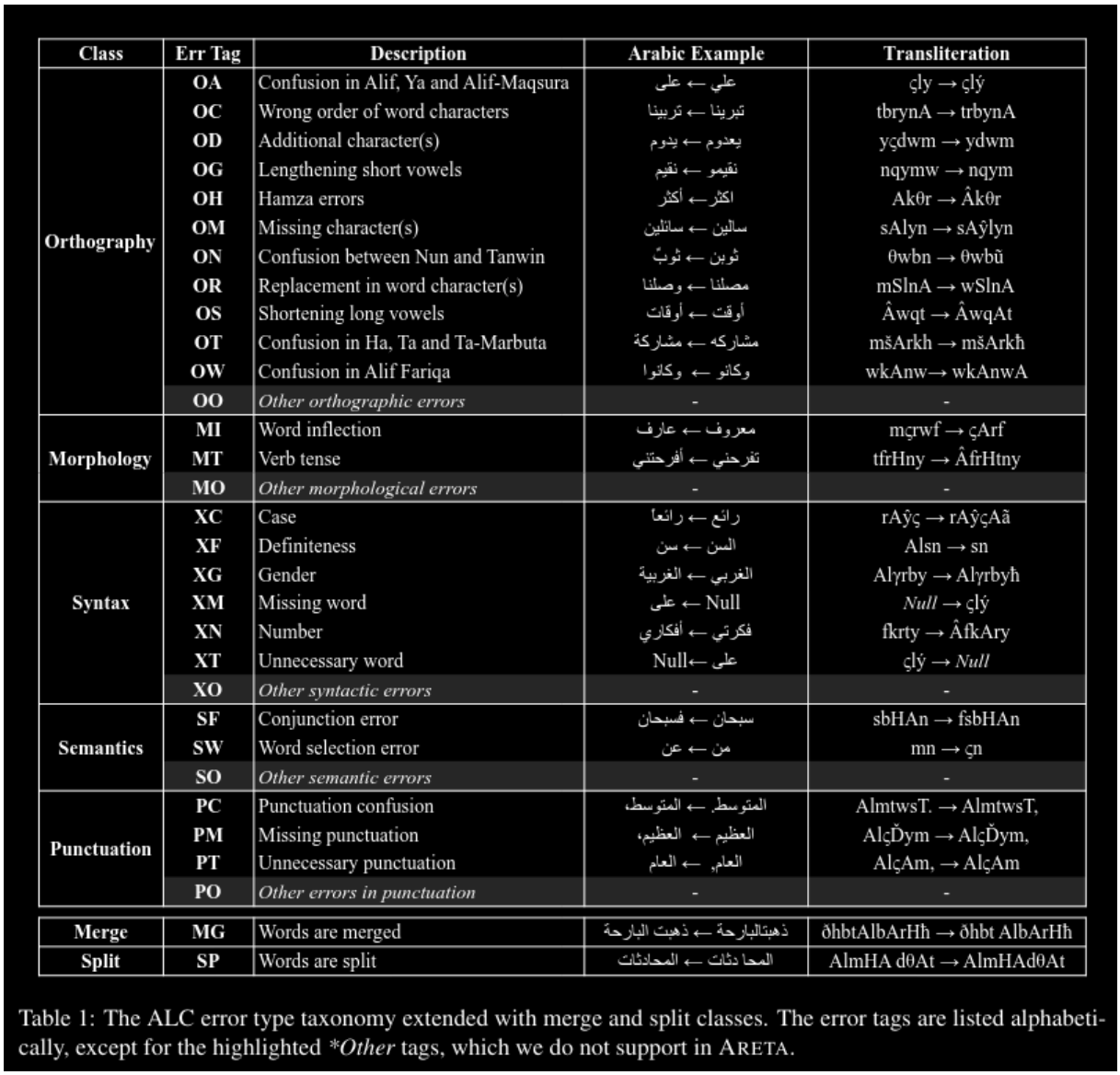
Table 3.1: *Comparison of Related Arabic Writing Assistance Approaches*

Work/System	Main Approach	Strengths	Limitations
SAHSOH@QALB-2015 [5]	Rule-based correction on QALB	Explainable and lightweight	Limited coverage
Moukrim et al. [14]	Ontology-based syntactic correction	Strong grammar grounding	Hard to scale
Solyman et al. [15]	Transformer-based Arabic GEC	Strong benchmark results	Heavier and less interpretable
Alhafni et al. [16]	Pretrained seq2seq GEC with GED input	State-of-the-art performance	Model-heavy pipeline
Alhafni and Habash [17]	Text-editing GEC	Fast and accurate	Needs integration with full product layers
ARWI [19]	Arabic writing assistant workflow	Closest system-level reference	Learner-oriented scope

3.4. Implemented Approach

The survey suggests that Baligh should not depend on a single method. A purely rule-based system is easy to explain but limited in coverage, while a purely neural system is stronger statistically but harder to interpret and deploy. Baligh therefore adopts a hybrid design that combines linguistic preprocessing, explainable correction logic, and lightweight predictive assistance.

At the front of the pipeline, Baligh uses morphology-aware preprocessing because segmentation, normalization, and disambiguation are essential for Arabic NLP [1, 3]. For correction, the literature supports a layered strategy: symbolic rules and ontology-inspired ideas remain useful for precision and explanation, while modern GEC findings, especially text-editing approaches, offer better scalability and speed [5, 14, 17].



Class	Err Tag	Description	Arabic Example	Transliteration
Orthography	OA	Confusion in Alif, Ya and Alif-Maqsura	علي ← على	ɕly → ɕlý
	OC	Wrong order of word characters	تبرينا ← تربينا	tbrynA → trbynA
	OD	Additional character(s)	يعدوم ← يدوم	yɕdwm → ydwm
	OG	Lengthening short vowels	نقيمو ← نقيم	nqymw → nqym
	OH	Hamza errors	اكثر ← أكثر	Akθr → Âkθr
	OM	Missing character(s)	سالين ← سالتين	sAlyn → sAŷlyn
	ON	Confusion between Nun and Tanwin	ثوبن ← ثوبٌ	θwbn → θwbũ
	OR	Replacement in word character(s)	مصلنا ← وصلنا	mSlnA → wSlnA
	OS	Shortening long vowels	أوقات ← أوقات	Âwqt → ÂwqAt
	OT	Confusion in Ha, Ta and Ta-Marbuta	مشاركه ← مشاركة	mšArkh → mšArkħ
	OW	Confusion in Alif Fariqa	وكانو ← وكانوا	wkAnw→ wkAnwA
	OO	*Other orthographic errors*	-	-
Morphology	MI	Word inflection	معروف ← عارف	mɕrwf → ɕArf
	MT	Verb tense	تفرحني ← أفرحتني	tfrHny → ÂfrHtny
	MO	*Other morphological errors*	-	-
Syntax	XC	Case	رائع ← رائعاً	rAŷç → rAŷçAã
	XF	Definiteness	السن ← سن	Alsn → sn
	XG	Gender	الغربي ← الغربية	Alɣrby → Alɣrbyħ
	XM	Missing word	على ← Null	*Null* → ɕlý
	XN	Number	فكرتي ← أفكاري	fkrty → ÂfkAry
	XT	Unnecessary word	Null← على	ɕlý → *Null*
	XO	*Other syntactic errors*	-	-
Semantics	SF	Conjunction error	سبحان ← فسبحان	sbHAn → fsbHAn
	SW	Word selection error	من ← عن	mn → ɕn
	SO	*Other semantic errors*	-	-
Punctuation	PC	Punctuation confusion	المتوسط، ← المتوسط.	AlmtwsT. → AlmtwsT،
	PM	Missing punctuation	العظيم ← العظيم،	AlɕĎym → AlɕĎym،
	PT	Unnecessary punctuation	العام، ← العام	AlɕAm، → AlɕAm
	PO	*Other errors in punctuation*	-	-
Merge	MG	Words are merged	ذهبتالبارحة ← ذهبت البارحة	ðhbtAlbArHh → ðhbt AlbArHh
Split	SP	Words are split	المحا دثات ← المحادثات	AlmHA dθAt → AlmHAdθAt

Table 1: The ALC error type taxonomy extended with merge and split classes. The error tags are listed alphabetically, except for the highlighted **Other* tags, which we do not support in ARETA.

Figure 3.2: *Taxonomy of common Arabic grammatical and orthographic errors relevant to Baligh.*

The same reasoning applies to prediction. Since Baligh is interactive, efficient language modeling is a practical baseline for next-word suggestion, with neural models reserved for cases where extra context modeling justifies the cost [11, 12]. Separating detection, correction, and explanation is also consistent with the literature because it improves interpretability and makes user feedback more informative [4, 16].

In summary, Baligh follows the literature by combining morphology-aware preprocessing, structured error handling, explainable correction support, and efficient prediction into one real-time writing workflow.

Chapter 4: System Design and Architecture

This chapter forms the core of the report, detailing the technical foundation of Baligh. It outlines the design principles and structural components that enable the system to perform high-accuracy Grammatical Error Correction (GEC) and Next-Word Suggestion (NWS) with real-time responsiveness. Beginning with the high-level system architecture and then diving deep into the specific inner workings of each module and submodule.

4.1. Overview and Assumptions

Baligh is designed as a comprehensive Modern Standard Arabic (MSA) writing assistant that aims to provide real-time Grammatical Error Correction (GEC) and Next-Word Suggestion (NWS). The primary goal is to achieve high linguistic accuracy combined with low latency to ensure a seamless typing experience. To achieve this, the system is architected to differentiate between slow-path operations (such as deep grammatical checking) and fast-path operations (such as real-time predictive text).

During the design and implementation of the system, several key assumptions and technical constraints were established:

- **Joint Morphological Processing:** Diacritization and morphological analysis are mutually dependent in Arabic. The system assumes that morphological disambiguation implicitly provides the correct diacritization based on context, meaning they are resolved in a single joint pass rather than sequentially.
- **Real-Time Word Boundary Handling:** Since user input is instantaneous and often contains incomplete words at the cursor, the system assumes that only tokens terminated by specific delimiters (whitespace or punctuation) are fully formed. Incomplete fragments bypass the full morphological pipeline and are directly routed to the word auto-completion engine.
- **Modularity for Explainability:** The system assumes that users need understandable feedback. Thus, error detection and correction are loosely coupled, allowing the Explanation/Fusion layer to compile clear reasoning from multiple detection subsystems (rule-based, pattern-based, and ML).

4.2. System Architecture

The system architecture of Baligh is highly modular, orchestrating data through a carefully constructed pipeline that balances deep linguistic analysis with immediate user responsiveness. The architecture is broadly separated into three main layers: the Client Layer, the Preprocessing Pipeline, and the Core Features Layer.

Data enters the system through the **Client Layer**, which consists of the user interface (web text editor) and the REST API backend. The raw text is immediately sent to the **Preprocessing Pipeline**. This pipeline performs foundational text cleaning, normalization, tokenization via Farasa, and contextual morphological analysis and disambiguation using CAMeL tools.

Once the text is preprocessed, it is dispatched into the **Core Features Layer**, which splits into two parallel tracks:

1. **The Slow Path (GEC & GED):** Preprocessed text is fed into the Grammatical Error Detection (GED) Engine, which utilizes a fusion of rule-based checkers, pattern matchers, and machine-learning sequence labelers to identify errors. These findings are passed to the Grammatical Error Correction (GEC) Module, which proposes corrections via an ontology-guided grammar lookup, dictionary spelling checks, and a text-editing machine learning model. Each module generates human-readable explanations. A final fusion and ranking layer consolidates the corrections and explanations, ensuring that the user receives the most relevant and understandable feedback.
2. **The Fast Path (NWS):** For predictive typing, the Next-Word Suggestion (NWS) module operates with minimal latency. It relies on a smart caching layer (for idioms and user patterns) and a context-aware trie-based lookup for word autocompletion (WAC) or next-word prediction (NWP).

4.2.1. Block Diagram

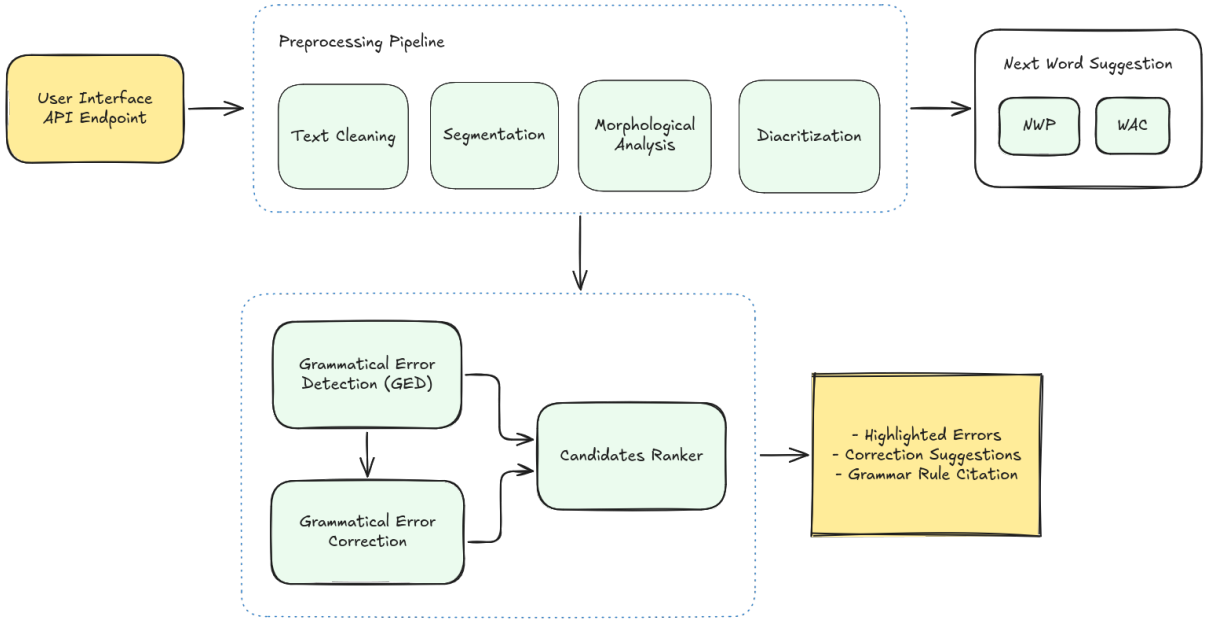


Figure 4.1: *High-level modular architecture of the Baligh system.*

4.3. Module 1: Preprocessing Pipeline

The Preprocessing Pipeline is the entry point of the Baligh backend, responsible for parsing, cleaning, and extracting linguistic features from raw text before downstream Grammatical Error Detection (GED), Correction (GEC), or Next-Word Suggestion (NWS) modules are invoked.

4.3.1. Unicode Normalization and Character Mapping

To ensure consistent processing, raw text typed by users goes through a normalization and cleaning stage. The team designed and implemented this pipeline using the following steps:

- **Unicode NFKC Normalization:** Normalizes the input text using the Unicode NFKC standard to canonicalize varying character codepoints and ligature representations into standard forms.
- **Text Cleaning and Tatweel Stripping:** Strips redundant tatweel character segments (ـ) and consolidates consecutive whitespaces into single spaces to remove orthographic noise.
- **Offset Mapping Array Construction:** Builds a mapping array, `norm_to_orig_map`, where index i in the normalized string maps back to the corresponding character index in the original, unnormalized user input.

4.3.2. Word Boundary Detection and Mode Selection

In a real-time editor, text is received continuously. Downstream GED/GEC checkers must only run on completed words, while the NWS module must decide whether to complete the active word fragment or predict the next word. To address this, a boundary splitting algorithm analyzes the trailing character of the normalized input, resulting in one of two outcomes:

- **Word Auto-Completion (`"WAC"`) Mode:**
  - **Condition:** The trailing character is an Arabic letter or digit, indicating the user is in the middle of typing a word.
  - **Behavior:** The last whitespace-delimited chunk is split off and extracted as the `current_fragment`. The remaining preceding text is treated as the completed prefix and passed to the segmentation and morphological analysis pipeline.

- **Next-Word Prediction (`"NWP"`) Mode:**
  - **Condition:** The trailing character is a delimiter (such as a whitespace or a punctuation mark like ،, ؟, or a period), indicating the last word is complete.
  - **Behavior:** The `current_fragment` is set to null. The entire input is treated as completed tokens and parsed to serve as context for predicting the next word.

4.3.3. Tokenization and Affix Segmentation

The core goal in this stage was to use the Farasa segmenter to identify morphological components (prefixes, stems, and suffixes) while ensuring that Farasa's aggressive Arabic NLP engine does not alter, fix, or ruin the raw user text intended for downstream Grammatical Error Detection (GED) modules. Because GED relies on identifying spelling and layout errors directly from the user's raw input, any implicit corrections or structural modifications performed by Farasa would mask errors and prevent their detection.

To resolve this conflict, the team designed and implemented a series of custom bypass mechanisms to handle the following issues encountered during integration:

1. **Implicit Spelling Corrections:**
   - **The Problem:** Farasa acts as an automatic spell-checker. When a user types a word with a spelling mistake, such as missing a Hamza (اكرم instead of أكرم) or a Ta-Marbuta typo (مدرسه instead of مدرسة), Farasa silently corrects the token in its output. Consequently, the downstream GED module would receive the corrected forms and fail to detect the user's typo.
   - **Resolution Strategy:** A character-by-character mapping bridge was implemented using Python's `difflib.SequenceMatcher`. Before alignment, the algorithm standardizes characters (such as neutralizing Hamza variants and Ta-Marbuta/Ha differences) on both the raw and Farasa-processed strings to prevent alignment mismatches. The algorithm then reconstructs the segmented tokens by pulling the exact characters typed by the user at each position, effectively restoring the user's spelling mistakes while keeping Farasa's segmented boundaries intact.
2. **Aggressive Word and Number Splitting:**
   - **The Problem:** When users omit space boundaries (e.g., typing اكرم100 as a single block), Farasa splits the input into two separate, grammatically

valid tokens: أكرم and 100. This hides spacing errors, making it impossible for the GED module to flag a missing space.

- **Resolution Strategy:** Farasa was stripped of its authority to define word boundaries by enforcing a regex-first tokenization pass. A regex pattern was designed to capture contiguous blocks of Arabic letters, digits, and diacritics as single tokens, while isolating punctuation and whitespace. If this regex pass groups a segment as a single token but Farasa splits it, the orchestrator merges the pieces back together, overrides Farasa's output, and sets the token's `affix_structure` to null. This enables downstream modules to correctly flag it as an error.

3. **Dropped Diacritics (Tashkeel):**

   - **The Problem:** Farasa frequently strips Arabic short vowels (diacritics) during segment analysis (e.g., segmenting المنَزل as ال + منزل or لَأكلها as ل + أكل + ها). Additionally, standard Python regular expressions (`\w`) do not capture Arabic diacritics as word characters, causing words to break.
   - **Resolution Strategy:** First, the regex tokenization pattern was updated to explicitly include the Unicode ranges for all Arabic diacritics. Second, a lookahead alignment algorithm was built in the token reconstruction loop. If a diacritic present in the raw text is missing from Farasa's segmented output, the algorithm retrieves the diacritic character from the raw text and dynamically re-injects it before the plus clitic boundary (yielding لَ + أكل + ها).

4. **Inserted and Hidden Characters:**

   - **The Problem:** Farasa sometimes inserts letters that do not exist in the raw text to reconstruct clitics grammatically. For instance, the token للحج is segmented as ل + ال + حج, inserting an implicit Alif (ا) that was not typed by the user.
   - **Resolution Strategy:** The character-mapping bridge was designed to handle unmapped characters. When the reconstruction loop encounters an inserted character in Farasa's output that has no index in the raw text, it accepts Farasa's inserted character and continues the mapping sequence without crashing or dropping tokens.

After reconstructing the spelling-preserved segmented tokens, the clitics are identified by peeling prefixes and suffixes. Prefixes are matched against conjunctions (و, ف), prepositions (ب, ل, ك), and definite articles (ال). Suffixes are matched against subject/object pronouns (e.g., ها, هم, نا, ه). The remaining core is rejoined as the `STEM`, and

the sequence of clitics is stored as a plus-joined string in the token's `affix_structure` attribute.

4.3.4. Morphological Analysis and Disambiguation

To provide downstream layers with high-fidelity grammatical and syntactic metadata, the pipeline implements a joint analysis and contextual disambiguation process:

- **Candidate Feature Generation:** Each completed token is processed through CAMeL Tools' morphological analyzer to generate a comprehensive list of potential analyses. Each candidate captures lexical and morphological properties, including lemma, Part-of-Speech (POS) tag, gender, number, person, case, aspect, and diacritized form.
- **Contextual Disambiguation:** Because written Arabic is highly homographic (often omitting short vowels), a single token can have multiple valid morphological analyses out of context. To resolve this ambiguity, the system applies CAMeL Tools' sequence-based disambiguator, which analyzes the surrounding sentence context to identify the single most probable candidate.
- **Downstream API Integration:** The chosen disambiguated candidate is marked with the flag `is_disambiguated=true` and positioned at index 0 of the token's morphological candidate array. This establishes a clean interface for downstream GEC and GED modules, allowing them to retrieve the correct context-aware features immediately.

4.4. Module 2: Grammatical Error Detection

The Grammatical Error Detection (GED) module is the primary analysis engine responsible for identifying linguistic issues in preprocessed Arabic text. It receives normalized tokens and morphological features from the preprocessing pipeline and produces a unified set of annotated error spans, each enriched with a category label, confidence score, and optional explanation. The GED module follows a multi-detector architecture in which three complementary subsystems run in parallel before a fusion layer reconciles their outputs into one consistent result set.

4.4.1. Functional Description

The GED module operates on the output of the preprocessing pipeline, which supplies segmented tokens, part-of-speech tags, morphological features, and disambiguation

results. Its primary function is to identify spans of text that contain grammatical, orthographic, punctuation, or semantic errors and to assign each span an error category from the Baligh tag schema. Because the module is designed for an educational writing assistant, it also attaches human-readable explanations to rule-derived findings so that users can understand the reasoning behind each detection.

The module is invoked as part of the slow path in the Baligh architecture, meaning it runs after the user submits text for analysis rather than during keystroke-level interaction. This design allows the detectors to perform deeper linguistic checks without impacting the responsiveness of the fast-path next-word suggestion system.

4.4.2. GED Architecture

The GED subsystem is built around a layered detector architecture that combines symbolic and statistical approaches. Rather than relying on a single model family, the design separates concerns across three detector lanes and one fusion stage:

1. **Rule-Based Detector**: applies explicit linguistic checks using handcrafted rules.
2. **Lexicon Matcher**: detects curated lexical patterns, spelling anomalies, and split/merge errors using trie-backed resources.
3. **Machine-Learning Sequence Labeler**: predicts token-level GED labels using a trained CRF model with morphological features.
4. **Fusion Layer**: reconciles overlapping outputs from all three detectors into one final error set.

This hybrid strategy was motivated by the literature, which showed that symbolic systems remain useful in Arabic when interpretability is important, while learned approaches improve coverage on broader contexts [5, 14, 16].

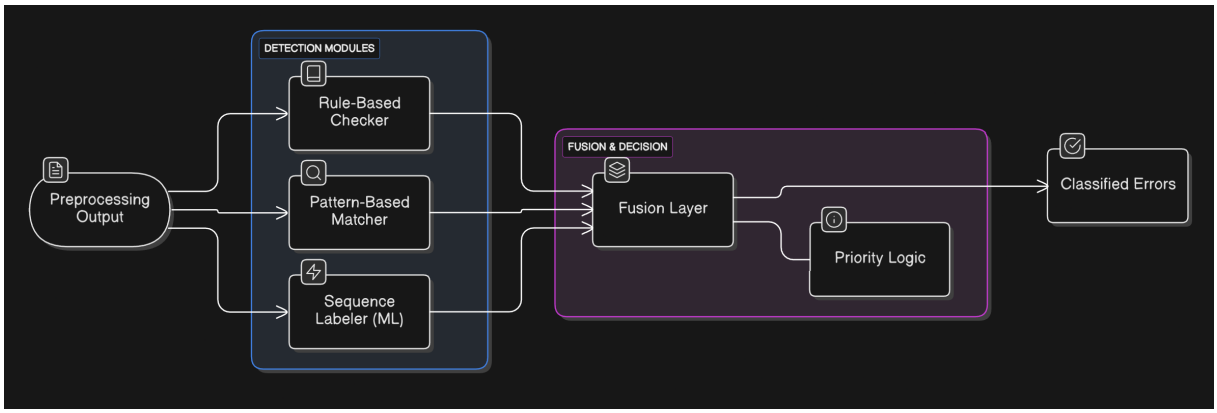


Figure 4.2: *GED detector architecture showing the rule-based, lexicon-based, and machine-learning detectors followed by the fusion and priority logic.*

4.4.3. Modular Decomposition

Rule-Based Subsystem

The rule-based detector is the most interpretable lane in the GED stack. It applies explicit linguistic checks to preprocessed text and produces high-confidence, explanation-friendly findings that can be verified deterministically. The subsystem supports a hybrid rule-hosting model:

- **Declarative YAML rules**: compact pattern-driven checks defined in structured configuration files, well-suited for token-matching and simple morphological conditions.
- **Procedural Python rules**: richer checks that require multi-token context, complex conditionals, or access to morphological feature combinations that cannot be expressed declaratively.

Both rule types are routed through a common registry that handles loading, validation, and execution. The rule authoring workflow involved sourcing grammar material from LanguageTool and online Arabic grammar references, drafting rules in the internal format, manually reviewing each rule against the tag schema and preprocessing assumptions, and testing them individually before acceptance.

The implemented rule base covers four categories: orthography rules (Hamza placement, Alif Maqsura, Ta Marbuta, Idgham patterns), punctuation rules (spacing and Latin-to-Arabic character variants), syntax rules (demonstrative-noun agreement, jussive operators, verb-subject agreement, preposition case requirements), and semantics rules (lexical usage preferences and stylistic recommendations). The full catalog is documented in Appendix D.

Lexicon Subsystem

The lexicon subsystem extends detection beyond procedural rules by combining curated token and split/merge patterns with trie-backed lookup and conservative spelling suspicion. Its role is to act as a middle layer between rigid symbolic rules and broader machine-learned predictions.

- **General word lexicon**: used to determine whether a normalized token is a plausible in-vocabulary Arabic word. Contains 9,009,550 entries.
- **Named-entity phrase lexicon**: used to protect people, places, organizations, and other entity mentions that may not appear in the general word list. Contains

60,833 phrases.

- **Named-entity token lexicon**: provides token-level entity coverage with 43,292 entries.

The word lexicon supports ordinary spelling lookup, while the entity lexicons reduce false positives on proper nouns and multi-token names. Additionally, the lexicon detector maintains curated split patterns (e.g., detecting when بعد ما should be written as بعدما) and merge patterns (e.g., detecting when انشاءالله should be split into إن شاء الله).

Table 4.1: *Lexicon Resources Used by the GED Lexicon Detector*

Resource	Size
General word lexicon	9,009,550 entries
Named-entity phrase lexicon	60,833 phrases
Named-entity token lexicon	43,292 tokens

Machine-Learning Subsystem

The machine-learning subsystem treats GED as a token-level sequence-labeling task over the Baligh error categories. The final deployed model is a morphology-aware Conditional Random Field (CRF) trained on QALB14 and QALB15 data, covering 19,706 sentences and 1,063,125 tokens.

Feature engineering. The CRF model uses a rich feature set organized into four groups:

- **Surface features**: normalized token form, token shape, token length, Arabic / digit / punctuation flags, prefixes and suffixes up to length four, sentence-boundary markers, neighboring normalized tokens, sentence-length and position buckets.
- **Orthographic cues**: diacritic presence, repeated-character indicators, definite-article cues, and character n-grams.
- **Morphology features**: POS, UD POS, lexeme, stem, root, pattern, gender, number, person, aspect, voice, mood, state, case, proclitic and enclitic fields.
- **Confidence and context features**: backoff-source indicators, discretized morphology log-probability buckets, and neighboring POS features.

Model selection. Multiple model architectures were evaluated during development, including token-memory heuristics, calibrated token memory, linear classifiers, and

CRF sequence models. The final selected model was the morphology-enhanced CRF (`crf_surface_v1_morph_v1`), which delivered the strongest development performance.

Table 4.2: *ML Detector Selection Summary*

Role	Model	Threshold	Precision	Recall	F1
Earlier candidate	Surface-only CRF	0.35	0.9080	0.8070	0.8545
Final deployed	CRF + morphology	0.40	0.9158	0.8332	0.8725

The final runtime detector is loaded from a model bundle with an inference manifest that keeps the training-time feature contract aligned with deployment.

Fusion Layer

The fusion layer is essential because a multi-detector system is only useful if overlapping outputs can be reconciled consistently. The implemented algorithm performs the following steps:

1. Sort all detected spans by start position and length.
2. Perform a single left-to-right sweep over the accepted list.
3. Apply resolution rules for exact-span duplicates, containment, partial overlap, and tier mismatches.
4. Normalize explanation eligibility so that statistical predictions do not claim rule-like explanations they do not possess.

The tier system assigns priority levels: Tier 1 rule-derived detections have the highest confidence and can override statistical detections on the same span, while Tier 3 statistical detections provide broader coverage for spans not addressed by symbolic detectors.

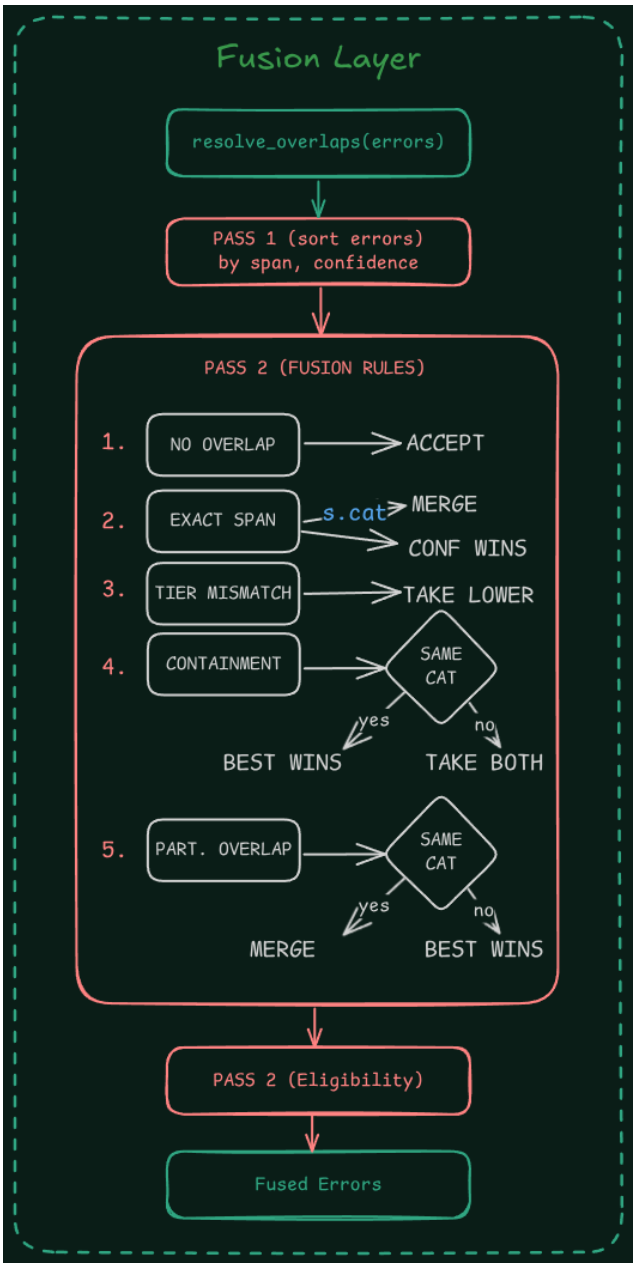


Figure 4.3: *Fusion-layer flow used to resolve overlap conflicts and normalize final GED spans.*

Table 4.3: *Implemented GED Subsystems and Their Roles*

Subsystem	Purpose	Main Strength	Implementation Approach
Rule-based detector	Apply linguistic checks over pre-processed text	Precision and explainability	Hybrid YAML/Python rule hosting with registry
Lexicon matcher	Detect curated lexical and spelling-oriented issues	Bridge between rules and statistical prediction	Trie-backed resources with pattern matching
Sequence labeler	Predict token-level GED labels from features	Broad contextual coverage	CRF with morphology-aware feature engineering
Fusion layer	Merge subsystem outputs into one final result set	Stable multi-detector behavior	Tier-based overlap resolution with explanation gating

4.4.4. Design Constraints

Several constraints shaped the design of the GED module:

- **Arabic morphological ambiguity**: Arabic tokens are highly ambiguous due to the absence of short vowels in ordinary writing, clitic attachment, and context-dependent grammatical role assignment. The GED module depends on reliable morphological disambiguation from the preprocessing pipeline to produce meaningful detections.
- **Explainability requirements**: Because Baligh targets educational use, GED outputs must carry human-readable explanations rather than opaque labels. This constraint motivated the inclusion of rule-based and lexicon-based detectors alongside the statistical component.
- **Data scarcity**: High-quality labeled Arabic GED data is limited compared with English resources. The CRF model was trained on QALB14 and QALB15, and the rule base was developed to extend coverage to error types underrepresented in training data.
- **Latency tolerance**: The GED module operates on the slow path, meaning it runs after the user submits text rather than during typing. This allows deeper analysis but requires that the combined detector and fusion pipeline completes within a reasonable response window.

4.4.5. Evaluation Summary

The GED module was evaluated across multiple corpora including QALB14, QALB15 (L1 and L2), and ZAEBUC. The full per-corpus breakdown is presented in Chapter 5 (Table 5.1). The aggregate binary results demonstrate that the symbolic lanes contribute precise but narrower coverage, while the learned and fused systems deliver stronger end-to-end performance.

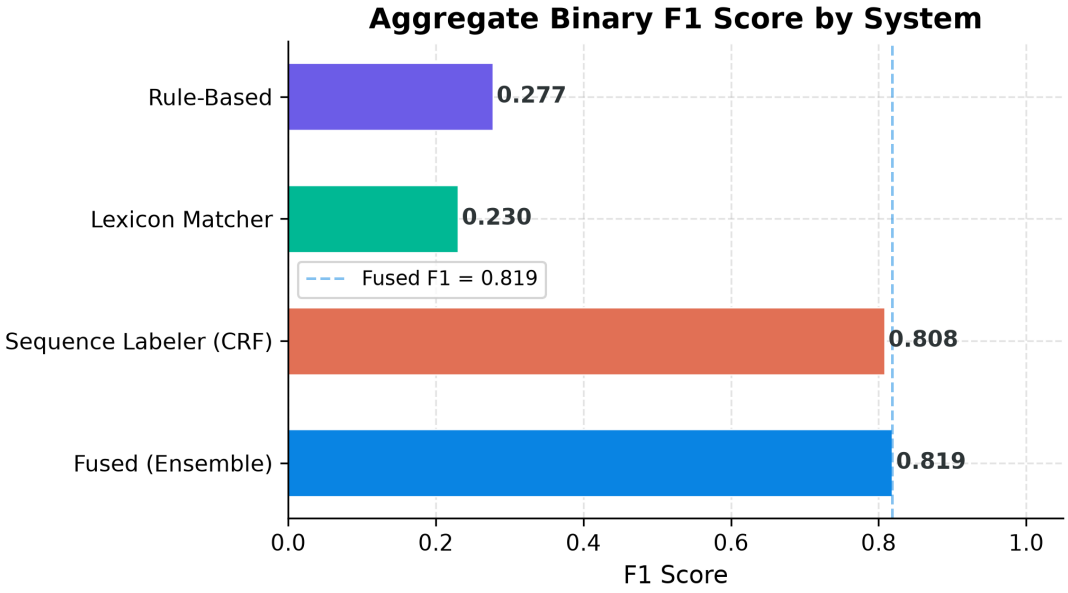


Figure 4.4: *Aggregate binary GED F1 score by subsystem from the evaluation results.*

Table 4.4: *Aggregate Binary GED Performance Across All Evaluation Corpora*

System	Precision	Recall	F1
Rule-Based	0.685	0.174	0.277
Lexicon Matcher	0.892	0.132	0.230
Sequence Labeler	0.913	0.725	0.808
Fused	0.905	0.747	0.819

These numbers should be interpreted with care because the error categories are not evenly distributed across the evaluation datasets. In practice, the datasets highlight models that perform well on the more frequent tags, which naturally favors the sequence labeler. However, the rule-based and lexicon-based lanes still detect error types that the sequence labeler may miss, especially categories that are less common in the training data. The fused gain demonstrates that symbolic detectors contribute useful coverage beyond the dominant dataset patterns.

4.4.6. GED Output in the GUI

Because Baligh is ultimately a user-facing writing assistant, the detector output must be understandable in the editor rather than only measurable in an evaluation script. Figure 4.5 shows two real GUI captures: one emphasizing highlighted detections inside the editor text, and one showing the issue card and explanation-oriented presentation attached to a detected span.

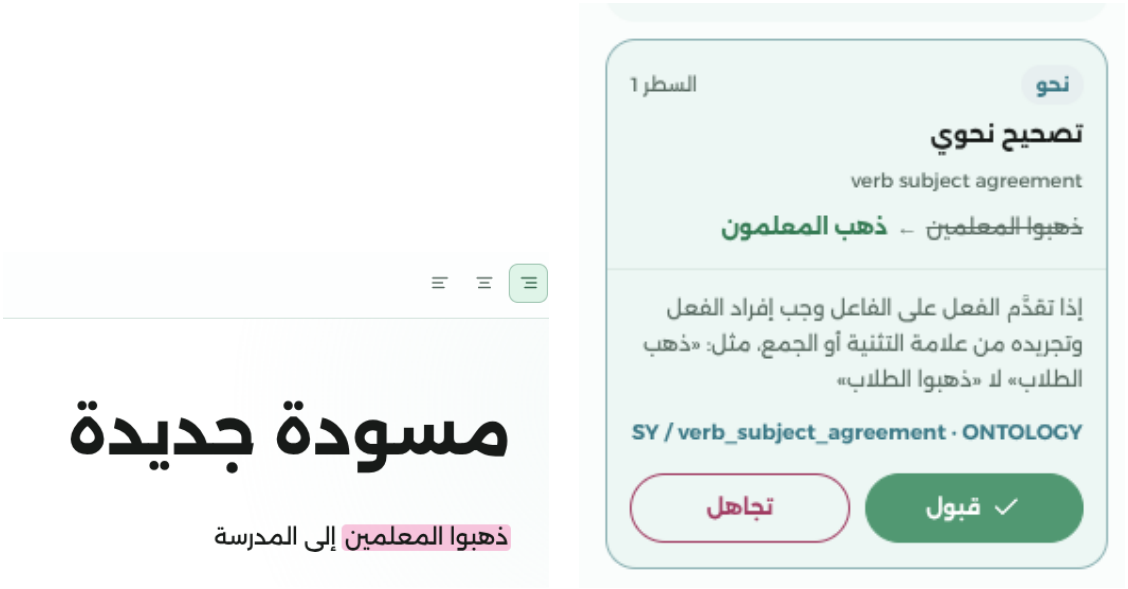


Figure 4.5: *GED feedback inside the Baligh editor: highlighted detections in the writing view and an issue card showing explanation-oriented feedback for a detected error.*

4.5. Module 3: Correction and Explanation Generation

The Grammatical Error Correction (GEC) module processes text to generate accurate correction candidates for grammatical and spelling mistakes. This module is divided into three primary components: the Dictionary Module, the Ontology Module, and the Edit Tagger Module.

4.5.1. The Dictionary Module

Functional Description

The Dictionary Module is responsible for robust spelling correction, Out-Of-Vocabulary (OOV) detection, and serving as a morphological generation backbone for the ontology. The Arabic language presents unique challenges in lexical validation due to its

highly inflectional nature. To address this, the module utilizes the **Arramooz** dictionary, an open-source comprehensive Arabic lexicon organized in relational database tables. Arramooz encapsulates approximately 6,000 roots and covers stop words, nouns, and verbs.

The complete dictionary pipeline involves detecting OOV words by stripping clitics (prefixes/suffixes) to isolate the stem, querying the database for close string matches, and retrieving spelling candidates.

Modular Decomposition

The Dictionary Engine consists of the following submodules:

- **Spelling Checker:** Analyzes the incoming tokens and performs stem-based clitic analysis. Then it detects Out-Of-Vocabulary (OOV) words and generates a list of spelling candidates using edit distance metrics.
- **Arramooz Client:** A wrapper designed for efficient, low-latency querying of the SQLite-based Arramooz database.
- **Morphological Generator:** Acts upon requests from the Ontology Engine. When a specific inflection is required (e.g., converting a singular noun to a plural form in the accusative case), this generator queries Arramooz to construct the morphologically correct word form.
- **Alternative Ranker:** Ranks spelling candidates using Levenshtein distance (edit distance) combined with corpus-frequency metrics to present the most probable correction first.

Design Constraints

The primary constraint of the Dictionary Module is the reliance on raw string similarity (edit distance) for initial spelling suggestions. Edit distance lacks context-awareness. To counter this, the Dictionary Engine relies on downstream contextual rankers (like the Machine Learning fusion layer) to make the final decision among its proposed alternatives.

Results

Figure 4.6: *Spelling Error (Original)*

Figure 4.7: *Dictionary Correction*

Figure 4.8: *Typographical Error (Original)*

Figure 4.9: *Dictionary Correction*

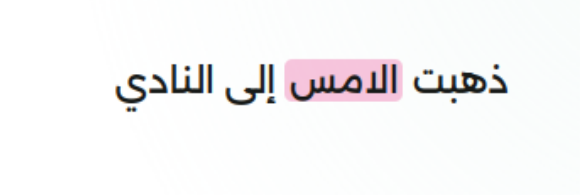


Figure 4.10: *Lexical Error (Original)*

Figure 4.11: *Dictionary Correction*

4.5.2. The Ontology Module

Functional Description

An **Ontology** in computer science is a formal, explicit specification of a shared conceptualization. It defines a set of concepts and categories in a subject area, showing their properties and the relations between them. In the context of Baligh, the Ontology Module encodes Arabic syntax rules into a computational format, inspired by dependency grammar based on predicate logic.

This approach leverages **Semantic Web** technologies. The semantic web operates on layers, primarily the Resource Description Framework (RDF), which represents data as subject-predicate-object triples. For Arabic grammar, the **Ontology of Arabic Syntax (OAS)** is modeled using **OWL** (Web Ontology Language), an extension of RDF that allows for highly expressive logic and reasoning. In the OAS, concepts represent grammatical categories (e.g., Noun, Verb) while relationships represent syntactic links (e.g., Subject-Verb dependency) and functional properties (e.g., Case, Gender).

The methodology employed is “Generation then Comparison”:

1. **Categorization:** Words are assigned Lexical Features (meaning-heavy aspects like Transitivity) and Functional Features (structural aspects like Number, Gender, Case).
2. **Merger (Generation):** Using **SPARQL** (the query language for RDF), the system merges words into oriented grammatical pairs. By querying the OAS, the system generates all syntactically correct combinations of the sentence that abide by Arabic grammar axioms.
3. **Comparison:** The original incorrect sentence is compared against the generated valid sentences using string distance metrics, allowing the system to deduce the required correction.

Modular Decomposition

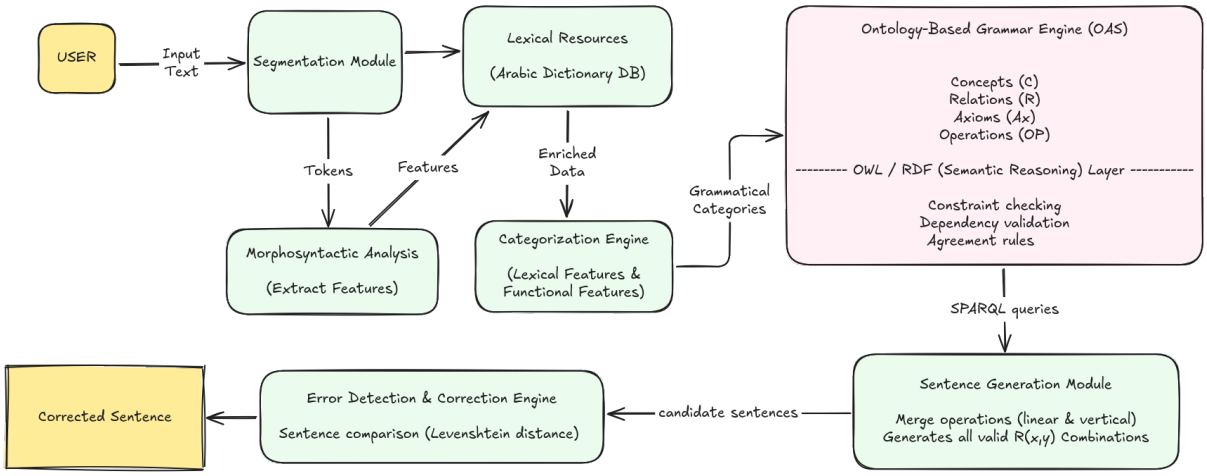


Figure 4.12: *Architecture of the GEC Module.*

The Ontology Engine orchestrates the generation logic via the following components:

- **Feature Mapper:** Translates CAMeL morphological outputs into the specific OWL terminology required by the OAS.
- **SPARQL Query Generator:** Constructs dynamic semantic queries. For example, it can query the ontology to enforce the axiom that a Verb's Subject must possess a Nominative case.
- **Candidate Generator:** Combines the theoretical rules from the ontology with the Morphological Generator from the Dictionary to produce the final corrected string phrase.
- **Explanation Generator:** Utilizes the specific violated ontology axiom to dynamically construct an educational, human-readable grammatical explanation for the user.

Design Constraints

Generating every possible syntactically correct permutation of a full sentence is computationally prohibitive, posing a significant latency risk for a real-time system. To circumvent this, the generation phase is aggressively localized. Instead of processing the entire sentence, the Ontology Engine restricts its generation only to the precise error spans previously identified by the Grammatical Error Detection (GED) module. Furthermore, bridging the terminology gap between the morphological analyzer's tags and the OAS concepts required building a rigid, robust mapping dictionary to prevent query failures.

Results

Figure 4.13: *Grammatical Error (Original)*

Figure 4.14: *Ontology Correction*

Figure 4.15: *Syntactic Mismatch (Original)*

Figure 4.16: *Ontology Correction*

Figure 4.17: *Case/Gender Error (Original)*

Figure 4.18: *Ontology Correction*

4.5.3. Text Editing Sub-module

Functional Overview

The system implements a Grammatical Error Correction (GEC) model for Arabic using a **text editing (sequence tagging)** approach rather than a traditional sequence-to-sequence (Seq2Seq) generative model. By treating correction as a sequence of edit operations rather than generating text from scratch.

Correction Generation

Instead of generating the corrected sentence autoregressively, the model predicts a sequence of edit operations for each token in the erroneous input. These predicted edits are then applied to the original text to produce the corrected sentence.

- **Edit Extraction & Alignment:** The system aligns erroneous and corrected sentence pairs using a weighted Levenshtein edit distance, accompanied by an iterative algorithm to capture multi-word errors. It then applies the character-level alignments.
- **Applying Operations:** It assigns specific character-level edits to the input words to transform them. The primary operations include:
  - **Keep (K)**
  - **Delete (D)**
  - **Insert (I_[])**

  - **Replace (R_[])**

- **Optimization Strategies:** To keep the vocabulary of possible edits manageable and improve learning, the system uses:
  - **Edit Compression:** Merging consecutive operations (e.g., consecutive deletes `DDD` into `D*` or insertions into `I_[ ] I_[*]`).
  - **Sub-word Projection:** Edit operations are first extracted at the word level, where grammatical corrections are naturally defined. These edits are then projected onto the corresponding sub-word tokens so that each token receives a label, matching the tokenization scheme used by Transformer models such as BERT.
  - **Edit Segregation:** Splitting the correction into a two-step pipeline—one model for non-punctuation errors, and a subsequent model for punctuation errors.
  - **Edit Pruning:** Removing rare edit operations during training to help the model focus on frequent and informative edits.

Explanation Generation (Interpretability)

Unlike autoregressive Seq2Seq models, which generate only the final corrected sentence, edit-based models explicitly predict the edit operation applied to each input token (e.g., **KEEP**, **DELETE**, **REPLACE**, or **INSERT**). These edit labels provide a transparent record of how the input was transformed, making it straightforward to identify the exact grammatical changes performed. This improves the interpretability of the correction process and facilitates error analysis, debugging, and the generation of user-friendly explanations for the applied corrections.

Modular Decomposition

- **Alignment Module**
  The aligner is responsible for aligning the erroneous sentence with its corrected one using the Levenshtein edit distance algorithm. The alignment is performed in two stages. First, alignment is carried out at the **word level** to determine the correspondence between words in the two sentences. At this stage, operations such as **insertions**, **deletions**, **merges**, and **splits** are identified to handle cases where a single word in one sentence corresponds to multiple words in the other, or vice versa. Once the word-level alignment has been established, each aligned word pair is further aligned at the **character level** to determine the exact character insertions, deletions, and replacements required to transform the

erroneous word into its corrected form. This hierarchical alignment provides the detailed edit information that serves as the basis for extracting edit labels used during model training.

- **Rewrite Module**
  After the initial alignment, the rewriter generates an intermediate representation of the aligned sentences. This step resolves ambiguities in the character-level alignment and rewrites complex edits into a consistent form that can be more easily projected onto tokens.
- **Edit Projection Module**
  Since grammatical edits are initially extracted at the character level while Transformer-based models operate on tokenized input, the projector maps the character-level edit operations onto the corresponding tokens or sub-word tokens. Each token is assigned an edit label describing the operation that should be applied, making the extracted edits compatible with the model's token-level prediction mechanism.
- **Edit Compression Module**
  The projector may produce multiple consecutive edit operations for a single token. The compressor combines these individual operations into a compact edit label whenever possible. This reduces redundancy in the label sequence, simplifies the classification problem, and produces a more manageable set of edit tags for model training and inference.
- **Edit Pruning Module**
  The pruner analyses the frequency of all generated edit labels across the training corpus and removes edit labels that occur below a predefined frequency threshold. Rare edit operations contribute little to the learning process while significantly increasing the number of output classes. Pruning these infrequent labels reduces label sparsity, decreases vocabulary size, and allows the model to focus on learning the most common and informative correction patterns.
- **Edit Segregation Module**
  Punctuation corrections often exhibit different characteristics from grammatical and spelling corrections. The segregator separates punctuation-related edit labels from non-punctuation edits, allowing them to be processed independently. This separation improves the handling of punctuation corrections and enables the system to treat punctuation edits differently during training or inference if required.
- **Edit Application Module**
  During inference, the model predicts a sequence of edit labels rather than generating corrected text directly. The edits applicator interprets these predicted

labels and applies the corresponding operations—such as insertions, deletions, replacements, merges, or splits—to the original input tokens. The resulting sequence of modified tokens is then reconstructed to produce the final corrected sentence.

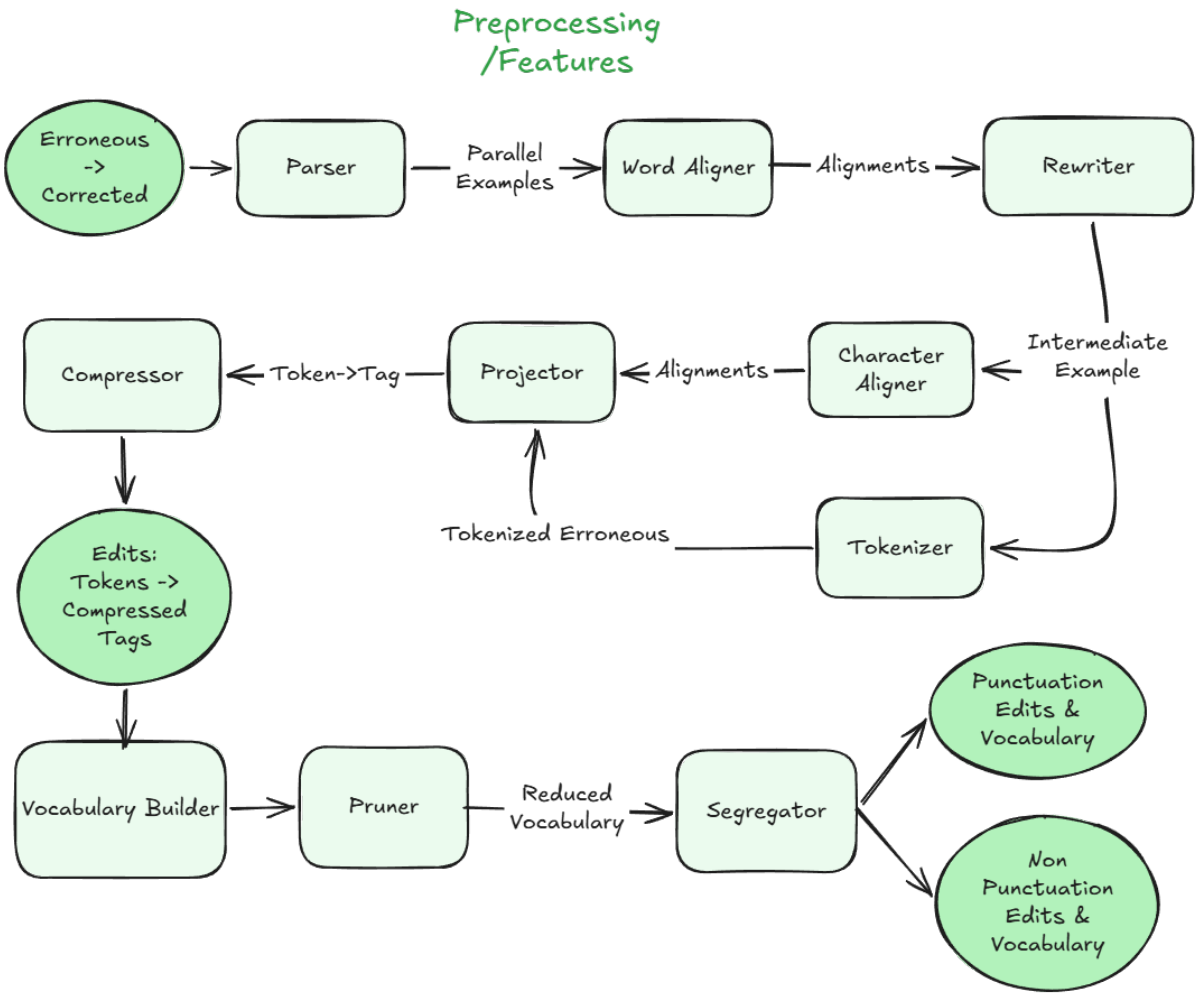


Figure 4.19: *Tagger Pre-processing Pipeline*

Design Constraints

- The edit tagging task exhibits a significant class imbalance, as the majority of tokens in the training corpus do not require correction and are therefore assigned the KEEP label. This imbalance may bias the model toward predicting KEEP more frequently than edit operations, potentially reducing its ability to detect less common grammatical errors.
- The frequency of grammatical error types in the training corpus is highly uneven. Errors related to Hamza (إ,أ,ء), Taa' Marbuta (ه,ة), and punctuation constitute a large proportion of the annotated corrections, whereas many other grammatical

phenomena occur much less frequently

These corpus characteristics directly influence the model's learning behavior. Frequent edit operations are learned more reliably due to their abundance in the training data, while infrequent grammatical phenomena remain more challenging to detect and correct.

Results

Figure 4.20: *Edit Tagger Correction Example (2)*

Figure 4.21: *Edit Tagger Correction Example (1)*

Figure 4.22: *Edit Tagger Correction Example (4)*

Figure 4.23: *Edit Tagger Correction Example (3)*

4.5.4. Ranker Sub-module

Functional Description

The Ranker module is responsible for producing the final coherent text alongside its supporting grammatical explanations.

- **Correction Generation** evaluates the corrections coming from GEC making use of the GED output. It applies a rigorous filtering and scoring pipeline to candidates from multiple modules. Overlapping or conflicting candidate edits are resolved by selecting the candidate with the highest composite score.
- **Explanation Generation** is facilitated by hardcoded scoring biases that favour explainable, rule-based corrections. Specifically, if a detected error is marked as `explanation_eligible` and the candidate correction comes from the rule-based `ONTOLOGY` module, a positive weight (`W_EXPLAIN`: +0.05) is applied. This bias guarantees that the final selected corrections can be mapped back to transparent linguistic rules rather than relying solely on black-box predictions.

Modular Decomposition

This module is decomposed into four primary submodules:

1. **Configuration Manager**
   Manages the tunable hyper-parameters used during scoring. Key configurations include base module weights (`W_ONTOLOGY`: 0.25, `W_TAG`: 0.15, `W_DICTIONARY`: 0.10), confidence weightings (`W_EDIT_CONF`: 0.30), and penalty weights (`W_CHAR_DIST`: 0.10, `W_LOW_CONF`: 0.15).
2. **Candidate Aggregator & Matcher**
   Maps candidate edits to specific errors detected by the GED by checking span intersections. Candidates whose correction equals the original text are immediately filtered out.
3. **Scoring Engine**
   Computes a comprehensive `score` for each candidate edit using multiple weighted heuristics:
   - **Base Module & Confidence**: Adds base module weight, edit confidence (`W_EDIT_CONF`), and GED confidence (`W_GED_CONF`).
   - **Provenance Tiers**: Boosts scores for `tier_1_rule_derived` (+0.15) and `tier_2_rule_supported` (+0.08) error spans.

- **Linguistic Alignment**: Awards +0.10 (`W_SPELL_DICT`) if a Dictionary module corrects an `ORTHOGRAPHY` error, and +0.10 (`W_GRAM_ONT`) if the Ontology module corrects a `SYNTAX` or `MORPHOLOGY` errors. Out-of-vocabulary (`is_oov`) tokens receive a +0.05 bonus.
- **Multi-Source Agreement**: Awards +0.20 (`W_AGREEMENT`) for each peer module that suggests the exact same correction string.

4. **Final Selection & Metadata Generator**
   Sorts the scored candidates per error using a strict tuple key: `(score, edit_confidence, module_priority, -length_of_correction)`. `module_priority` is strictly ranked: Ontology (3) > Tag (2) > Dictionary (1). It then iterates chronologically through the text, applying the best candidate. Finally, it emits the final correction.

Design Constraints

- **Grammatical Accuracy and Precision**:
  The system penalizes edits that diverge too much from the original text. A penalty (`W_CHAR_DIST`: -0.10) is applied proportionately to the normalized Levenshtein distance, and another penalty (`W_LENGTH_RATIO`: -0.05) is applied for deviations in string length. Furthermore, if a candidate's `edit_confidence` falls below `CONF_LOW_THRESHOLD` (0.30), a flat penalty of -0.15 (`W_LOW_CONF`) is applied.

4.6. Module 4: Next-Word Suggestion (NWS)

The NWS subsystem runs in parallel with the main GEC/GED layers as a fast-path service. Figure 4.24 shows the architecture of the NWS module.

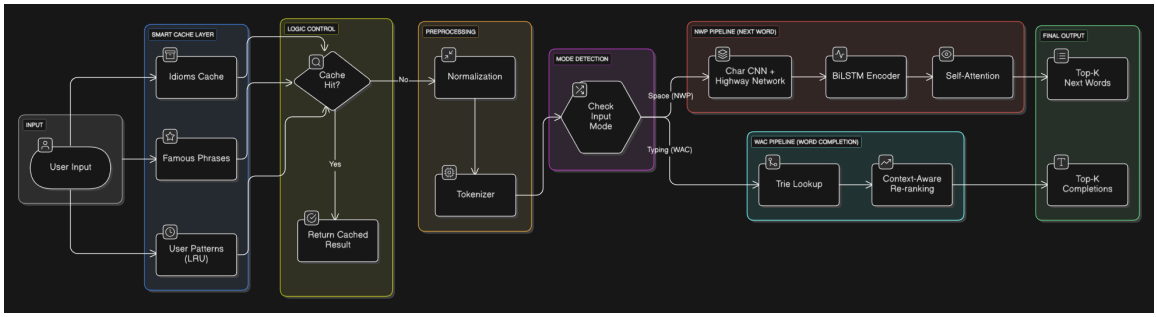


Figure 4.24: *Architecture of the Next-Word Suggestion (NWS) and Auto-Completion subsystem.*

4.6.1. Three-Tier Caching Architecture

To achieve sub-millisecond latencies, a three-tier cache was designed to intercept requests before model evaluation:

- **Tier 1 (Idioms Cache):** A static cache loaded from a YAML configuration containing common Arabic idioms and fixed expressions.
- **Tier 2 (Famous Phrases Cache):** A static cache storing common collocations and religious/opening formulas.
- **Tier 3 (User Patterns Cache):** A dynamic LRU cache that stores successful predictions generated by the models to speed up repeat requests.

If a cache key (built from the last N tokens) hits any tier, the suggestions are returned immediately.

4.6.2. Next-Word Prediction (NWP) Subsystem

To build a strong next-word prediction system (mode `"NWP"`), the system combines a statistical language model with a neural sequence model. To compare them fairly and avoid domain mismatch, a unified dataset called `final_corpus` was created using high-quality text from Arabic Wikipedia and the Mixed-Arabic Dataset. All models (LSTM, Word N-Gram, and Char N-Gram) were trained on this same dataset. Its sizes are shown in Table 4.5.

Table 4.5: *Dimensions of the `final_corpus` dataset used for training and evaluating NWS models.*

Split	Sentences	Total Words (Tokens)	Total Characters
Train	653,332	17,055,048	172,061,726
Validation	33,420	906,414	9,087,105
Test	34,233	899,776	9,051,354
Total	**720,985**	**18,861,238**	**190,200,185**

The NWS pipeline generates predictions by combining the following subsystems:

- **Word-Level N-Gram Predictor:**
  - **Architecture:** When a prediction is not found in the cache, the system checks a statistical word N-gram language model (`WordNGramLM`) using Modified Kneser-Ney (MKN) smoothing. This model calculates the probability of the next word based on the previous words.
  - **The Scaling Experiment Limit:** The team tested how increasing the training dataset size affects this model's accuracy using Arabic Wikipedia. Even after growing the dataset to 41 million words, the Top-5 accuracy stopped improving at 18.2%. This showed a hard accuracy limit, proving that simple counting-based N-Gram models lack the deep context understanding needed to cross the 20% accuracy mark, because they only look at the most recent words.
  - **Hindawi E-Books Dataset Baseline:** The model was tested on the Hindawi E-Books dataset (∼1,745 books) using two different setups, as shown in Table 4.6.

Table 4.6: *Baseline evaluation on the Hindawi E-Books dataset. The hybrid setup performed worse due to candidate recall limitations.*

Setup	Top-1 Accuracy	Top-5 Accuracy	MRR
Standalone KenLM (50K unigrams)	7.6%	18.2%	0.114
Hybrid (Pruned N-Gram + KenLM)	-	14.6%	-

- **LSTM Language Model:**
  - **Architecture:** To understand long-term context, an LSTM language model (`ArabicLSTMLM`) was added. Its structure includes a 12,000-word subword

vocabulary built with SentencePiece, an embedding layer, dropout to prevent overfitting, a 2-layer LSTM core with 512 hidden units, and an output layer. Predictions are generated efficiently using beam search decoding.

- **Training Metrics:** Training this 6.88M parameter model on the `final_corpus` gave the results shown in Table 4.7.

Table 4.7: *Standalone LSTM training metrics and final evaluation on the `final_corpus`.*

Training Stage	Perplexity (PPL)	Top-5 Accuracy
Epoch 1	152.01	-
Epoch 10 (Final)	91.00	21.52%

- **Confidence-Weighted Hybrid Blending:**
  - **Combining Scores:** The system combines the neural LSTM predictions and the statistical MKN N-gram predictions using a confidence-weighted score:

$$\text{Score}_{\text{final}}(w) = \alpha \cdot \log P_{\text{LSTM}}(w \mid C) + (1 - \alpha) \cdot \log P_{\text{MKN}}(w \mid C)$$

  - **Dynamic Weights:** The mixing weight α changes automatically. If the LSTM is very confident (score $>$ log(0.35)), it is trusted more ($\alpha = 0.75$). Otherwise, they are balanced equally ($\alpha = 0.50$). Words predicted only by the statistical model get a penalty of -20.0, and final scores are converted to probabilities using softmax.
  - **Final Hybrid Results:** By mixing the LSTM's deep understanding with the Word N-Gram's ability to memorize common phrases, the final combined model reached the accuracy shown in Table 4.8.

Table 4.8: *Final hybrid model evaluation metrics showing synergistic improvements.*

Metric	Accuracy	Improvement vs. Standalone LSTM
Top-1 Accuracy	11.40%	-
Top-3 Accuracy	18.20%	-
Top-5 Accuracy	23.00%	+1.48%

In conclusion, training both the statistical model and the neural network on the exact same dataset (`final_corpus`) fixed domain differences and created a highly effective predictor. The Word N-Gram model successfully memorized common phrases to help cover the LSTM's weaknesses, proving that the hybrid approach works very well.

4.6.3. Word Auto-Completion (WAC)

When the system detects that the user is actively typing a word (mode `"WAC"`), it predicts the rest of the word using the following steps:

- **Trie Matching:** To find possible word completions, the system first searches a MARISA-Trie containing the entire dictionary. This quickly finds all valid words that start with the exact letters the user just typed. By only using words from this trie, the system is prevented from generating fake or invalid Arabic words.
- **Character N-Gram Predictor:** To score these found words, a character-level N-gram model (`CharNGramLM`) is used instead of a word-level model. This choice solves the Out-Of-Vocabulary (OOV) problem. A word model cannot score a word if it was never seen during training. Since the Arabic language has countless word forms and variations, a character-level model is needed to reliably score any word by looking at its letters, even if that specific word variation was never seen in the training data.
- **Length-Normalized Scoring:** The raw scores of the candidate words are adjusted based on their length to ensure the system doesn't unfairly prefer very short or very long words. The scoring formula is:

$$\text{Score}(W) = \frac{\log P(W \mid C)}{|W|^{0.7}}$$

  where $|W|$ is the length of the candidate word.

The evaluation of the character N-gram model for word auto-completion yields the overall performance metrics presented in Table 4.9.

Table 4.9: *Overall evaluation metrics for the Word Auto-Completion (WAC) character N-gram model.*

Metric	Score
Top-1 Accuracy	30.00%
Top-3 Accuracy	40.80%
Top-5 Accuracy	46.40%
Mean Reciprocal Rank (MRR)	0.3699
Keystroke Savings Rate (KSR)	7.40%

4.6.4. System Suggestions

To illustrate the functionality of the Next-Word Suggestion (NWS) and Word Auto-Completion (WAC) subsystem, Figure 4.25 shows an example of active auto-completion word suggestions, and Figure 4.26 presents examples of next-word predictions generated dynamically within the user editor interface.

Figure 4.25: *Example of Word Auto-Completion (WAC) active word suggestions in the user interface.*

Figure 4.26: *Examples of Next-Word Prediction (NWP) suggestions generated at a word boundary.*

4.7. Module 5: Web Client

The web client is the user-facing application that exposes Baligh's capabilities through an interactive writing environment. Built with React and Vite, it provides a lightweight, design-system-aligned interface that communicates with the backend API to present analysis results, corrections, and suggestions to the user.

4.7.1. Functional Description

The web client serves two primary functions. First, it provides a public-facing landing experience that introduces Baligh and its capabilities. Second, it offers an editor workspace where users write Arabic text and receive real-time feedback from the GED, GEC, and NWS modules. The editor highlights detected issues inline, displays issue cards with explanations, and surfaces correction candidates and next-word suggestions through contextual UI elements.

The client is structured around reusable, design-system-aligned components that follow an Arabic-first, learning-oriented design philosophy. The interface prioritizes clarity over visual complexity, ensuring that detector outputs appear as guided feedback rather than raw diagnostic logs.

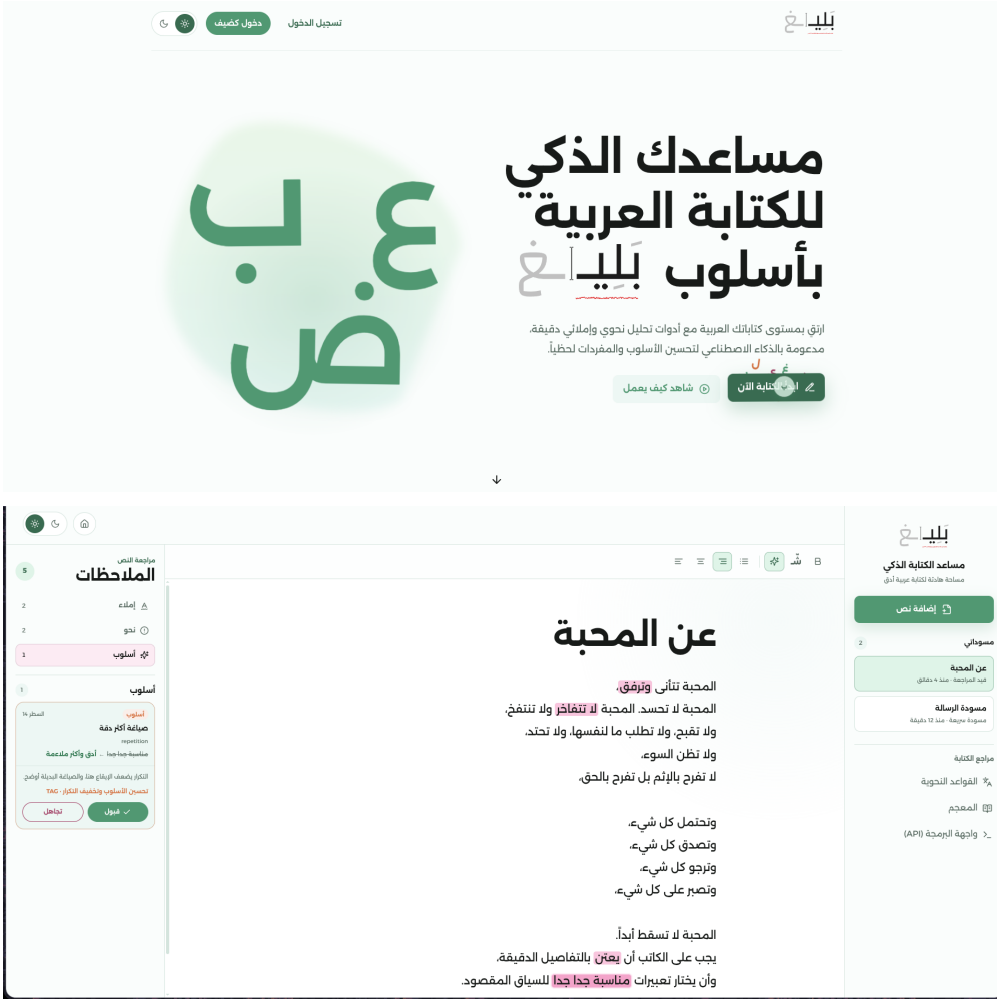


Figure 4.27: *The Baligh web client: the public landing page and the editor workspace used to present analysis results to the user.*

Chapter 5: System Testing and Verification

To ensure that Baligh operates correctly, achieves high linguistic accuracy, and maintains responsive real-time performance, a multi-layered testing and verification strategy was implemented. The validation framework includes automatic unit testing of isolated backend algorithms, end-to-end automated browser testing of frontend interactive elements, and evaluation of core Natural Language Processing (NLP) models on gold-standard Arabic corpora. This chapter details the testing environment setup, the modular and integrated testing methodologies, the continuous integration schedule, and the quantitative evaluation results of the grammatical error detection and next-word prediction modules.

5.1. Testing Setup

Testing and evaluation were conducted in a standardized environment to ensure the reproducibility of all results. The hardware and software specifications of the testing environment are defined as follows:

- **Operating System:** Linux (Ubuntu `22.04` LTS / Debian-based distributions).
- **Python Environment:** Python `3.11.x`, managed using the `uv` package installer and virtual environment manager to guarantee deterministic dependency resolution.
- **Node.js Environment:** Node.js v`24.x`, utilizing the `pnpm` package manager (version `10.33.4`) for frontend dependency caching and management.
- **Linguistic Libraries and Resources:**
  - `camel-tools` (version `1.5.7+`) loaded with the Modern Standard Arabic morphology database (`morphology-db-msa-r13`) and the Maximum Likelihood Estimation disambiguator (`disambig-mle-calima-msa-r13`).
  - `farasapy` (version `0.1.1`) for initial text segmentations.
- **Testing Libraries:** `pytest` (version `8.3.5`) for Python backend testing; standard Node testing libraries for frontend unit tests; and `playwright` for automated end-to-end browser execution.

- **Evaluation Corpora:**
  - **Qatar Arabic Language Bank (QALB):** The QALB-2014 and QALB-2015 test datasets, which contain native and non-native Arabic comments extracted from Aljazeera articles, characterized by a high frequency of spelling and punctuation errors.
  - **ZAEBUC:** A bilingual Arabic-English student writing corpus used to evaluate detection and correction robustness in educational settings.
  - **NWP Corpus:** An Arabic text corpus combining Arabic Wikipedia articles and the MAD corpus (totaling 900,000 test-split words) for next-word prediction models.

5.2. Testing Plan and Strategy

The verification strategy relies on testing components in isolation (module testing) before testing their interactions in a fully assembled pipeline (integration testing).

5.2.1. Module Testing

Each subsystem in Baligh is validated independently using automated unit tests to guarantee functional correctness under diverse inputs.

- **Preprocessing Service:** Unit tests in the `tests/services/preprocessing/` directory verify normalizer rules (such as Alif and Ya normalization), sentence boundary detection, word segmenters, and morphological analyzers. For example, `test_analyzer.py` validates that out-of-vocabulary (OOV) tokens (e.g., typos like "الطلابب" containing an extra letter) are correctly flagged with an `is_oov = True` attribute to prevent cascading errors in downstream grammar checks.
- **Grammatical Error Detection (GED):**
  - *Rule-Based Subsystem:* Unit tests verify syntactic, semantic, punctuation, and orthographical rules. Files like `test_syntax.py` and `test_punctuation.py` check specific grammar patterns, such as subject-verb agreement or missing spaces before punctuation.
  - *Lexicon Subsystem:* Verified in `test_detector.py` and `test_trie_runtime.py` to ensure that dictionary words and entity phrases are loaded correctly into MARISA trie data structures for rapid OOV detection.
  - *Machine Learning Subsystem:* Verified in `test_features.py` and `test_detector.py` to ensure sequence labeling features are extracted and parsed correctly

by the CRF model.

  - *Fusion Layer:* Tested in `test_fusion.py` to ensure overlap resolution works according to defined tier priorities (ex., Tier `1` rules override Tier `3` statistical detections; overlapping spans of the same category are merged; duplicate sources are consolidated).

- **Grammatical Error Correction (GEC):** Evaluated in `tests/services/gec/`.
  - *Ontology Subsystem:* Tests morphological generation rules and syntactic corrections (such as gender and case endings) by querying semantic reasoning rules.
  - *Dictionary Subsystem:* Tests spelling candidate suggestion engines, edit-distance generators, and phonetic ranking matchers.
- **Next-Word Prediction (NWP):** Verified in `tests/services/nws/`.
  - *Next-Word Prediction (NWP) Subsystem:* Tests candidate word predictions generated from the LSTM model and the Word N-Gram model, validating their hybrid confidence blending.
  - *Word Auto-Completion (WAC) Subsystem:* Tests typing completions using the Character N-Gram model to verify MRR and KSR metrics.
  - *Prototyping & Orchestration:* Utilized interactive Jupyter Notebooks to visually evaluate beam search rankings and maintained a local orchestrator test (`test_orchestrator.py`), which was excluded from final submission to avoid requiring large CUDA library dependencies.
- **Frontend Application:** Frontend unit tests ensure utility functions, schema parsers, and editor states function correctly under standard input conditions.

5.2.2. Integration Testing

Integration testing validates that data flows correctly across the entire application stack.

- **GED Orchestrator Integration:** Tested in `test_ged_orchestrator.py`. This ensures that a raw Arabic sentence is properly normalized, segmented, analyzed morphologically, routed through all three detectors, and fused into a final list of resolved error spans.
- **Client-Server API Integration:** Verified by testing backend endpoints to ensure HTTP requests correctly accept Arabic payloads, apply the processing pipeline, and return JSON responses conforming to the API schemas.

- **End-to-End Browser Automation:** Using Playwright (`pnpm test:e2e`), automated Chromium browser instances simulate actual typing sessions. The E2E suite verifies that when a user types text in the editor, error spans are visually highlighted, suggestion dropdowns populate with correction candidates, and explanatory text bubbles appear beneath active cursors.

5.3. Test Schedule

Testing is integrated into the daily development workflow through three distinct stages:

1. **Local Pre-Commit Verification:** Code formatting (`ruff format`), linting checks (`ruff check`), and static type checking (`mypy`) are run on local changes prior to committing.
2. **Local Pre-Push Validation:** Developers run the complete check suite locally using a unified command (`make all`), which triggers formatting, linting, type-checking, and the full backend `pytest` test suite.
3. **Continuous Integration (CI):** Managed via GitHub Actions on every pull request and push to the `main` branch. The CI workflow splits verification into two parallel jobs:
    - `quality`: Sets up a Python `3.11` environment, installs astral `uv`, restores cached NLP datasets (Farasa and CAMeL Tools), overrides PyTorch with a CPU-only version to reduce runner overhead, downloads morphology databases, and executes formatting checks, linting, type checking, and unit tests.
    - `frontend`: Sets up Node.js `24` and pnpm, checks formatting and linting, runs frontend unit tests, installs Playwright dependencies, and executes end-to-end browser tests.

5.4. Comparative Results to Previous Work

The performance of Baligh's primary modules was quantitatively evaluated on established benchmarks.

5.4.1. Grammatical Error Detection (GED) Performance

The GED system was evaluated on QALB-2014, QALB-2015 (separate native L1 and non-native L2 sets), and ZAEBUC test sets. Performance is measured using Precision (P), Recall (R), F1-Score, and False Positives per 1000 tokens (FP/1000). The results are summarized in Table 5.1.

Table 5.1: *Grammatical Error Detection Subsystem Performance*

Dataset	System / Detector	Precision (P)	Recall (R)	F1-Score	FP/1000
QALB-2014	Rule-Based Detector	69.22%	18.76%	29.52%	18.88
	Lexicon Matcher	89.64%	13.83%	23.96%	3.62
	Sequence Labeler (ML)	92.91%	77.49%	84.50%	13.39
	Fused Orchestrator	**92.03%**	**79.75%**	**85.45%**	**15.63**
QALB-2015 L1	Rule-Based Detector	71.61%	20.34%	31.68%	17.38
	Lexicon Matcher	88.21%	14.97%	25.59%	4.31
	Sequence Labeler (ML)	92.47%	81.84%	86.83%	14.37
	Fused Orchestrator	**91.52%**	**84.30%**	**87.76%**	**16.84**
QALB-2015 L2	Rule-Based Detector	55.79%	7.59%	13.36%	13.57
	Lexicon Matcher	88.87%	9.48%	17.13%	2.68
	Sequence Labeler (ML)	82.67%	40.66%	54.51%	19.24
	Fused Orchestrator	**82.49%**	**42.74%**	**56.31%**	**20.47**
ZAEBUC	Rule-Based Detector	62.58%	19.93%	30.23%	33.41
	Lexicon Matcher	96.97%	8.92%	16.34%	0.78
	Sequence Labeler (ML)	87.63%	77.98%	82.52%	30.87
	Fused Orchestrator	**87.58%**	**79.09%**	**83.12%**	**31.46**
Aggregate	Rule-Based Detector	68.48%	17.39%	27.73%	17.95
	Lexicon Matcher	89.17%	13.22%	23.02%	3.60
	Sequence Labeler (ML)	91.29%	72.48%	80.81%	15.51
	Fused Orchestrator	**90.50%**	**74.73%**	**81.86%**	**17.59**

The empirical results show that while the rule-based and lexicon detectors exhibit high precision (especially the lexicon matcher on ZAEBUC at 96.97%), they suffer from low recall. The machine-learning sequence labeler provides the highest recall, and the fused orchestrator successfully integrates all subsystems to achieve the best overall F1-score across all test sets, raising the aggregate F1-score to 81.86%.

5.4.2. Next-Word Prediction (NWP) Performance

The Next-Word Prediction (NWP) models were evaluated on the test split of the LSTM Corpus. We compare the standalone LSTM model against a hybrid predictor that blends LSTM and Word N-Gram models (with confidence weight $\alpha = 0.75$). The evaluation metrics include Top-1, Top-3, and Top-5 prediction accuracy, as shown in Table 5.2.

Table 5.2: *Next-Word Prediction (NWP) Accuracy Comparison*

Model	Top-1 Accuracy	Top-3 Accuracy	Top-5 Accuracy
Standalone LSTM	10.88%	17.56%	21.52%
Hybrid Predictor	**11.40%**	**18.20%**	**23.00%**

The hybrid predictor outperforms the standalone LSTM model by leveraging high-frequency local word patterns from the N-Gram model alongside the long-range semantic representation of the LSTM.

5.4.3. Word Auto-Completion (WAC) Performance

The context-aware Character N-Gram model for Word Auto-Completion was evaluated under typing scenarios. It achieves a Top-1 accuracy of 30.00%, Top-3 accuracy of 40.80%, and Top-5 accuracy of 46.40%, with a Mean Reciprocal Rank (MRR) of 0.3699 and a Keystroke Savings Rate (KSR) of 7.40%. The performance improves significantly as the length of the typed prefix increases, as detailed in Table 5.3.

Table 5.3: *WAC Top-5 Accuracy by Typed Prefix Length*

Prefix Length (chars)	1	2	3	4	5	6+
Top-5 Accuracy	11.76%	32.79%	54.00%	77.50%	94.44%	100.00%

As the user types, the character prefixes restrict the possible lexical search space, causing the top-5 prediction accuracy to rise from 11.76% for a single character to 100.00% once 6 characters have been entered.

Chapter 6: Conclusions and Future Work

This chapter summarizes the overall journey of developing the Baligh project. It details the various challenges encountered during the research, design, and implementation phases, along with the strategies used to overcome them. Furthermore, it outlines the valuable experiences and technical skills gained by the team members. Finally, the chapter presents the overarching conclusions drawn from the project, highlighting its main features and limitations, and provides strategic directions for future work to expand and improve upon the current system.

6.1. Faced Challenges

Throughout the development of Baligh, the team encountered several research, technical, and integration challenges. To ensure a modular architecture and facilitate future updates, these challenges are categorized by the respective system modules.

6.1.1. Research Challenges

- **Next-Word Suggestion (NWS) Module:** One of the primary difficulties encountered during the research phase was the poorness of literature on Next-Word Suggestion for the Arabic language. The available research was found to be limited in quantity and often lacking in quality. To compensate for this gap and achieve reliable results, it was necessary to look beyond Arabic and review papers focusing on other morphologically rich languages that share similar structural complexities.

6.1.2. Technical and Integration Challenges

- **Next-Word Suggestion (NWS) Module:** Developing a real-time predictive text module posed significant challenges, primarily revolving around the trade-off between latency and accuracy, data scarcity, and poorly existing literature.
  - **Model Latency vs. Accuracy:** Balancing the fast response time required for typing with the context-awareness of neural models was difficult. This was done by implementing a hybrid approach that blends a fast Word N-Gram model with an LSTM model.

  - **Misleading Literature on N-Gram Performance:** Most papers claiming that simple N-Gram models are highly fit for NWS tasks often exaggerate their performance and use unrealistic numbers, which caused us to question our own results for a time. Furthermore, these studies assume the availability of massive corpora containing hundreds of millions to over a billion tokens to achieve good accuracy. Due to computing power limitations, training on such massive datasets was not feasible for our project.
  - **Data Scarcity for Interactive Evaluation:** While there are standard corpora for Arabic, finding datasets that closely mimic user keystrokes for evaluating Word Auto-Completion was challenging. We had to synthesize evaluation strategies using MRR and KSR metrics.

- **Grammatical Error Detection (GED) Module**
  - **High False Positives in Statistical Models:** Initial sequence labeling models often over-flagged correct Arabic structures. This was addressed by introducing a fusion orchestrator that prioritizes rule-based and lexicon-based detections over statistical models.
  - **Morphological Ambiguity:** Arabic's rich morphology often caused the system to misidentify word boundaries or grammatical roles. Integrating Farasa and CAMeL Tools for accurate morphological analysis prior to detection was crucial to resolving this.

- **Grammatical Error Correction (GEC) Module**
  - **Enormous Search Space:** Generating correction candidates for out-of-vocabulary words yielded too many possibilities. We overcame this by using dictionary lookups paired with phonetic and edit-distance ranking to surface only the most relevant suggestions.
  - **Complex Rule Maintenance:** Building an ontology-based correction module required extensive manual encoding of Arabic grammatical rules. Keeping this rule set manageable and efficient required careful software design.

- **Integration and Web Application**
  - **Pipeline Latency:** Running GED, GEC, and NWS sequentially for every user keystroke caused noticeable delays. We resolved this by optimizing the MongoDB client, separating analysis onto background threads, and making NWS requests independent from grammar checks.
  - **Rich Text Editor Alignment:** Ensuring that the highlight spans from the backend exactly matched the user's text on the frontend—especially after

whitespace modifications or partial edits—required precise index mapping and synchronization.

6.2. Gained Experience

Working on Baligh provided the team with extensive hands-on experience in both technical and non-technical domains. The collaborative nature of the project allowed team members to specialize while maintaining a unified system.

6.2.1. Technical Skills

- **Natural Language Processing (NLP):** The team gained profound insights into the linguistic differences of Arabic NLP. This included developing robust pipelines for morphological analysis, tokenization, and handling orthographic ambiguities. We also gained practical experience in extracting meaningful features for sequence labeling algorithms like CRFs and applying N-Gram structures effectively for predictive tasks.
- **Machine Learning (ML) & Data Science:** Extensive experience was gained in the entire ML lifecycle, from curating and preprocessing messy Arabic text corpora to training, evaluating, and fine-tuning models. We learned to balance accuracy and latency by evaluating different architectures, such as blending LSTMs with N-Grams, and measuring performance using metrics like MRR and KSR.
- **Frontend UI Design and Backend Integration:** Although heavily streamlined during development, constructing the interactive interface strengthened our skills in custom, handcrafted UI design. Setting up the core backend also provided practical experience in handling API integrations and orchestrating requests efficiently.

6.2.2. Soft Skills and Teamwork

- **Modular Collaboration:** Dividing the project into NWS, GED, GEC, and Integration taught the team how to define clear API contracts and interfaces, allowing parallel development without blocking one another.
- **Version Control and CI/CD:** Utilizing Git, GitHub Actions, and rigorous code review processes ensured code quality and seamless integration of independently developed modules.
- **Problem Solving and Research:** The project required translating academic re-

search into practical engineering solutions, bridging the gap between theoretical Arabic linguistics and applied software engineering.

6.3. Conclusions

The Baligh project successfully demonstrates that a comprehensive, real-time Arabic writing assistant is achievable by combining traditional linguistic rules with statistical and machine learning approaches. The final system integrates Grammatical Error Detection, Grammatical Error Correction, and Next-Word Suggestion to provide a better user experience.

Key Features achieved by the project include:

- A hybrid detection and correction pipeline that accurately addresses a wide range of Arabic grammatical and orthographic errors.
- A fast, context-aware next-word prediction and auto-completion engine that reduces typing effort.
- Educational, explainable feedback that helps users understand their mistakes rather than just blindly accepting corrections.
- A highly responsive web interface customized for the Arabic language.

Current Limitations of the system include:

- The reliance on word-level and character-level models in the NWS module, which can occasionally miss deep semantic context.
- The ontology and rule-based corrections, while explainable, are currently limited in coverage and may not handle rare or complex grammatical structures.
- The system is primarily optimized for Modern Standard Arabic (MSA) and may struggle with dialectal Arabic.

6.4. Future Work

To build upon the foundation established by Baligh, subsequent development can focus on scaling the system and incorporating more advanced technologies.

6.4.1. Next-Word Suggestion (NWS) Enhancements

- **In-Sentence Completions:** Expanding the suggestion capabilities to support completions anywhere within the sentence context, rather than strictly predicting the next word at the end of a sentence.
- **Extensive Model Benchmarking:** Experimenting with and benchmarking a wider variety of models to determine the highest achievable results and accuracies for Arabic text.
- **Domain-Specific Training and User Personalization:** Training models on higher volume datasets and providing distinct models customized for specific text domains (e.g., legal, medical, or casual text). The system could then dynamically switch to a model based on the specific domain of the user currently using the application.

6.4.2. Grammatical Error Detection and Correction Enhancements

- **End-to-End Seq2Seq Models:** Integrating an advanced sequence-to-sequence model alongside the text-editing approach to handle complex, multi-word structural corrections.
- **Expanded Ontology:** Broadening the rule-based ontology to cover deeper semantic errors and stylistic improvements, beyond basic syntax and orthography.
- **Contextual Spelling Correction:** Utilizing contextual embeddings (like AraBERT) to detect real-word spelling errors where the word is correctly spelled but incorrect in context.

6.4.3. System and Platform Extensions

- **Dialectal Support:** Expanding the datasets and models to support common Arabic dialects alongside MSA.
- **Browser Extension & Plugins:** Developing browser extensions and plugins for popular word processors (e.g., Microsoft Word, Google Docs) to make Baligh accessible across different platforms.
- **Offline Mode:** Packaging the core lightweight models (like N-Grams and MARISA Tries) for offline usage on mobile devices or desktop applications.

References

[1] O. Obeid et al., “CAMeL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing,” in *Proceedings of the 12th Language Resources and Evaluation Conference*, 2020, pp. 7022–7032.

[2] N. Habash, *Introduction to Arabic Natural Language Processing*. Morgan & Claypool Publishers, 2010.

[3] A. Abdelali, K. Darwish, N. Durrani, and H. Mubarak, “Farasa: A Fast and Furious Segmenter for Arabic,” in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*, 2016, pp. 11–16.

[4] R. Belkebir and N. Habash, “Automatic Error Type Annotation for Arabic (ARETA),” in *Proceedings of the 6th Arabic Natural Language Processing Workshop*, 2021, pp. 21–32.

[5] W. Zaghouani, T. Zerrouki, and A. Balla, “SAHSOH@QALB-2015 Shared Task: A Rule-Based Correction Method of Common Arabic Native and Non-Native Speakers’ Errors,” in *Proceedings of the ACL-IJCNLP 2015 Student Research Workshop*, 2015, pp. 147–153.

[6] N. Habash and D. Palfreyman, “ZAEBUC: An Annotated Arabic-English Bilingual Writer Corpus,” in *Proceedings of the 13th Language Resources and Evaluation Conference*, 2022, pp. 780–790.

[7] A. Alrehili and A. Alhothali, “Tibyan corpus: balanced and comprehensive error coverage corpus using ChatGPT for Arabic grammatical error correction,” *PeerJ Computer Science*, vol. 11, e2724, 2025.

[8] J. Hajič, O. Smrž, P. Zemánek, J. Šnaidauf, and E. Beška, “Prague Arabic Dependency Treebank: Development in Data and Tools,” in *NEMLAR International Conference on Arabic Language Resources and Tools*, 2004.

[9] N. Habash et al., “CAMeL Treebank: An Open Multi-genre Arabic Dependency Treebank,” in *Proceedings of the 13th Language Resources and Evaluation Conference*, 2022, pp. 450–459.

[10] D. Halabi, E. Fayyoumi, and A. Awajan, “I3rab: A New Arabic Dependency Treebank Based on Arabic Grammatical Theory,” *arXiv preprint arXiv:2007.05772*, 2020.

[11] K. Heafield, “KenLM: Faster and Smaller Language Model Queries,” in *Proceedings of the Sixth Workshop on Statistical Machine Translation*, 2011, pp. 187–197.

[12] F. S. Al-Anzi et al., “Revealing the Next Word and Character in Arabic: An Effective Blend of Long Short-Term Memory Networks and CNN,” *Applied Sciences*, vol. 14, no. 10498, 2024.

[13] A. Taher, “Arabic Next Word Prediction using BERT and CBOW: A Comparative Study,” *Journal of Arabic Computational Linguistics*, 2024.

[14] C. Moukrim, T. Abderrahim, E. H. Benlahmer, and T. Almalki, “An innovative approach to autocorrecting grammatical errors in Arabic texts,” *Journal of King Saud University - Computer and Information Sciences*, vol. 33, no. 8, pp. 972–982, 2021.

[15] A. Solyman, Z. Wang, Q. Tao, A. A. M. Elhag, R. Zhang, and Z. Mahmoud, “Automatic Arabic Grammatical Error Correction based on Expectation-Maximization routing and target-bidirectional agreement,” *Knowledge-Based Systems*, vol. 240, 108103, 2022.

[16] B. Alhafni, N. Inoue, C. Khairallah, and N. Habash, “Advancements in Arabic Grammatical Error Detection and Correction: An Empirical Investigation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 600–618.

[17] B. Alhafni and N. Habash, “Enhancing Text Editing for Grammatical Error Correction: Arabic as a Case Study,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.

[18] Y. Kim, Y. Jernite, D. Sontag, and A. M. Rush, “Character-Aware Neural Language Models,” in *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2016, pp. 2741–2749.

[19] E. Chirkunov, A. Al-Sabbagh, and N. Habash, “ARWI: Arabic Write and Improve — A Writing Assistant for Arabic Learners,” *arXiv preprint arXiv:2501.01234*, 2025.

[20] B. AlKhamissi, M. ElNokrashy, and M. Gabr, “Deep Diacritization: Efficient Hierarchical Recurrence for Improved Arabic Diacritization,” *arXiv preprint arXiv:2011.00538*, 2020.

[21] K. M. Almunirawi and R. S. Baraka, “Parsing Arabic Verbal Sentence Using Grammar Ontology,” *Journal of Engineering Research and Technology*, vol. 8, no. 2, 2021.

[22] W. Zaghouani, “Critical Survey of the Freely Available Arabic Corpora,” *arXiv preprint arXiv:1702.07835*, 2017.

Chapter A: Development Platforms and Tools

This appendix details the comprehensive technology stack, hardware, and datasets utilized throughout the development, testing, and deployment phases of the Baligh system.

Programming Languages

- **Python ($\geq$3.11):** The primary language for the core NLP backend, machine learning inference, and API orchestration.
- **TypeScript & JavaScript:** Used for developing the web frontend application and the browser extension.
- **HTML5 & CSS3:** Core web technologies structuring and styling the user interfaces.

Backend Frameworks and Libraries

- **FastAPI & Uvicorn:** High-performance ASGI framework and web server used for serving the RESTful API endpoints.
- **PyTorch ($\geq$2.4):** The deep learning framework powering the neural network models.
- **HuggingFace Transformers & Accelerate:** Utilized for loading, fine-tuning, and running transformer-based language models efficiently.
- **Scikit-Learn & sklearn-crfsuite:** Employed for traditional machine learning approaches and Conditional Random Field (CRF) sequence tagging.
- **CAMeL Tools & FarasaPy:** Specialized Arabic Natural Language Processing libraries used extensively for text preprocessing, tokenization, and morphological analysis.
- **Pydantic:** Used for strict data validation and serialization schemas across the API.

- **Motor:** The asynchronous Python driver utilized for non-blocking communication with MongoDB.
- **SPARQLWrapper:** A Python wrapper around SPARQL services used to interface with the Ontology of Arabic Syntax (OAS).
- **Marisa-Trie:** A static and memory-efficient Trie data structure library used for fast lexical lookups.

Frontend Frameworks and Libraries

- **React 19 & React Router DOM:** The core UI library and routing solution for building the interactive single-page application.
- **Vite:** A fast, modern build tool and development server for the frontend workspace.
- **TailwindCSS v4:** A utility-first CSS framework enabling rapid, responsive styling.
- **React Aria Components & Framer Motion:** Used to construct accessible, highly interactive, and smoothly animated UI components.
- **Lucide React:** Providing scalable vector iconography throughout the application interfaces.

Databases and Storage

- **MongoDB:** A NoSQL document database used for scalable data persistence (e.g., storing user profiles, document histories).
- **SQLite:** A lightweight, file-based relational database employed locally to house the Arramooz lexical dictionary and ontology relational data for rapid local queries.

Datasets

- **Qatar Arabic Language Bank (QALB):** The QALB-2014 and QALB-2015 test datasets served as the primary benchmark corpora. These datasets contain native and non-native Arabic comments extracted from Aljazeera articles and are characterized by a high frequency of authentic spelling and punctuation errors.
- **Zayed Arabic English Bilingual Undergraduate Corpus (ZAEBUC):** An annotated corpus containing essays written by university students, utilized to evaluate models on authentic learner errors.

- **Arabic Wikipedia:** Data extracted from Arabic Wikipedia was used for large-scale language modeling, vocabulary extraction, and providing general-domain context for the predictive models.

Development and CI/CD Tools

- **Version Control & Automation:** Git, GitHub, and GitHub Actions formed the backbone of the continuous integration and delivery pipelines.
- **Package Management:** `uv` was utilized for extremely fast Python dependency management and virtual environments; `pnpm` managed Node.js packages in the frontend monorepo.
- **Backend QA: Ruff** (linting and formatting), **MyPy** (static type checking), and **Pytest** (unit testing framework).
- **Frontend QA: ESLint** & **Prettier** (linting and formatting), **Vitest** (unit testing), and **Playwright** (end-to-end browser testing).
- **Task Automation:** GNU **Make** (via `Makefile`) and **pre-commit** hooks were used to orchestrate development workflows and enforce code quality standards before commits.

Chapter B: Use Cases

The Baligh system serves a diverse set of users by addressing their specific Arabic writing needs:

- **Arabic Language Learners:** To understand their grammatical mistakes through explainable corrections and improve their writing skills.
- **Content Creators & Journalists:** To quickly draft, proofread, and ensure their articles are free from linguistic errors before publication.
- **Teachers & Educators:** To assist in grading essays or preparing grammatically sound educational materials.
- **Lawyers & Professionals:** To draft formal documents, contracts, and emails with high linguistic accuracy and professionalism.
- **Companies & Institutions:** To maintain standardized, error-free Arabic communication across their platforms and internal tools.
- **Translators:** To ensure that content translated into Arabic is grammatically sound, flows naturally, and uses appropriate vocabulary.

Chapter C: User Guide

For a demonstration of how to use the Baligh system, please refer to our video demo: https://www.youtube.com/watch?v=1NjsZ_7A3k8

Additional details, including step-by-step instructions on how to install and run the project, can be found in the project's README repository: https://github.com/El-Maktab/baligh

Chapter D: GED Rules Reference

This appendix provides a comprehensive reference of the rule inventory that supports the implemented GED subsystem. The rules are organized by category: orthography, punctuation, syntax, and semantics. The lexicon split and merge patterns are listed separately. The rule-based catalog is maintained in the project documentation, while the split and merge patterns are defined in the lexicon detector resource files.

D.1. Rule-Based GED Catalog

D.1.1. Orthography Rules

Rule ID	Description
OT_HAMZA_PREP	حروف الجر والربط وبعض الأدوات التي أصلها بهمزة قطع تكتب بالهمزة لا بالألف المجردة، مثل: إلى، أو، إذا، أن، إن.
OT_ALIF_MAQSURA_ALA	الصواب «على» لا «علي».
OT_ALIF_MAQSURA_HATTA	الصواب «حتى» لا «حتي».
OT_TANWIN_NASB_ON_ALIF	تنوين النصب يكتب على الحرف الذي قبل الألف لا على الألف نفسها.
OT_IDGHAM_AN_MA	الأفصح إدغام «عن ما» في «عمّا» عندما تليها جملة فعلية.
OT_IDGHAM_MIN_MA	الأفصح إدغام «من ما» في «ممّا» عندما تليها جملة فعلية.
OT_TA_MARBUTA_NOUN	وجوب كتابة التاء المربوطة «ة» في أواخر الأسماء المؤنثة بدل الهاء «ه».
OT_TA_MARBUTA_ADJ	وجوب كتابة التاء المربوطة «ة» في أواخر الصفات المؤنثة بدل الهاء «ه».
OT_TA_MARBUTA_NOUN_PROP	وجوب كتابة التاء المربوطة «ة» في أواخر الأعلام المؤنثة بدل الهاء «ه»، مثل: مكة، فاطمة.
OT_INNA_AFTER_QAWL	همزة «إن» تُكسر وجوباً بعد القول.
OT_IDHA_HAMZA	«إذا» تُكتب بهمزة قطع تحت الألف.
OT_ARWAH_HAMZA	كلمة «أرواح» جمع تكسير تُكتب بهمزة قطع على الألف.
OT_AQAL_HAMZA	أفعل التفضيل «أقل» يُكتب بهمزة قطع على الألف.

D.1.2. Punctuation Rules

Rule ID	Description
PC_SPACE_BEFORE_PUNC	وجوب اتصال علامات الترقيم العربية مباشرة بالكلمة التي تسبقها دون فاصل أو مسافة.
PC_LATIN_COMMA_ARABIC	في السياق العربي تستعمل الفاصلة العربية «،» لا الفاصلة اللاتينية.
PC_LATIN_QUESTION_ARABIC	في السياق العربي تستعمل علامة الاستفهام العربية «؟» لا العلامة اللاتينية.
PC_LATIN_SEMICOLON_ARABIC	في السياق العربي تستعمل الفاصلة المنقوطة العربية «؛» لا العلامة اللاتينية المناظرة.

D.1.3. Syntax Rules

Rule ID	Description
SY_DEM_HADHANI_FEM	الصواب «هاتان» مع الاسم المثنى المؤنث لا «هذان».
SY_DEM_HATANI_MASC	الصواب «هذان» مع الاسم المثنى المذكر لا «هاتان».
SY_DEM_HADHAYNI_FEM	الصواب «هاتين» مع الاسم المثنى المؤنث لا «هذين».
SY_DEM_HATAYNI_MASC	الصواب «هذين» مع الاسم المثنى المذكر لا «هاتين».
SY_DEM_HADHANI_CASE_OBLIQUE_NOUN	إذا جاء بعد «هذان» اسم مثنى مذكر غير مرفوع فهناك خلل في المطابقة الإعرابية.
SY_DEM_HADHAYNI_CASE_NOM_NOUN	إذا جاء بعد «هذين» اسم مثنى مذكر مرفوع فهناك خلل في المطابقة الإعرابية.
SY_DEM_PREP_HADHAYNI_CASE_NOM_NOUN	بعد حرف الجر يكون الاسم بعد «هذين» مجرورًا لا مرفوعًا.
SY_DEM_HATANI_CASE_OBLIQUE_NOUN	إذا جاء بعد «هاتان» اسم مثنى مؤنث غير مرفوع فهناك خلل في المطابقة الإعرابية.
SY_DEM_HATAYNI_CASE_NOM_NOUN	إذا جاء بعد «هاتين» اسم مثنى مؤنث مرفوع فهناك خلل في المطابقة الإعرابية.
SY_DEM_PREP_HATAYNI_CASE_NOM_NOUN	بعد حرف الجر يكون الاسم بعد «هاتين» مجرورًا لا مرفوعًا.
SY_LAM_JUSSIVE	الفعل المضارع بعد «لم» يجب أن يكون مجزومًا.
SY_LAMMA_JUSSIVE	الفعل المضارع بعد «لما» يجب أن يكون مجزومًا.
SY_LA_NAHIYA_JUSSIVE	الفعل المضارع بعد «لا» الناهية يجب أن يكون مجزومًا.
SY_LA_NAFIYA_NOT_JUSSIVE	«لا» النافية لا تجزم الفعل المضارع.

Rule ID	Description
SY_INNA_SISTERS_DUAL_ACCUSATIVE	أخوات إن تنصب الاسم، والمثنى بعدها يكون منصوبًا بالياء لا مرفوعًا.
SY_VERB_SUBJECT_VSO	وجوب إفراد الفعل وتجريده من علامات التثنية والجمع إذا تقدم على الفاعل الظاهر في الجملة الفعلية.
SY_NOUN_ADJ_DEFINITENESS	وجوب مطابقة النعت للمنعوت في التعريف والتنكير.
SY_DEMONSTRATIVE_NOUN_GENDER	وجوب مطابقة اسم الإشارة للاسم الذي بعده في التذكير والتأنيث، مثل: هذا البطل، هذه السلامة.
SY_RELATIVE_PRONOUN_GENDER	وجوب مطابقة الاسم الموصول للاسم السابق له في التذكير والتأنيث، مثل: القول الذي، الوشاية التي.
SY_PREP_DUAL_CASE	وجوب جر الاسم المثنى بعد حرف الجر، مثل: في الكتابين لا في الكتابان.
SY_PREP_SOUND_MASC_PLURAL_CASE	وجوب جر جمع المذكر السالم بعد حرف الجر، مثل: مع المسافرين لا مع المسافرون.
SY_MAMNOO3_SARF_TANWEEN_NASB	الممنوع من الصرف لا يُنوَّن.
SY_KAMA_ANNE	الصواب «كما أن» بدون الواو الزائدة.
SY_LA_SIYYAMA_WA	الصواب «لا سيما» دون واو بعدها.
SY_AS_KA	استخدام الكاف للصفة استخدام أجنبي، والأصح النصب على الحالية.
SY_LA_PAST_TENSE	عند نفي الفعل الماضي يُنفى بـ «ما» أو «لم» مع المضارع، ولا يصح استخدام «لا» إلا إذا تكررت أو كانت للدعاء.
SY_MUBTADA_NAKIRA_PREP	المبتدأ النكرة يؤخر وجوباً إذا كان الخبر شبه جملة من جار ومجرور.
SY_QAAMA_BI	تعبير «القيام بـ» ركيك يستحسن تجنبه واستعمال الفعل مباشرة.
SY_HAWLA_FI	يفضل استعمال حرف الجر «في» بدلاً من «حول» في هذا السياق.
SY_RAGHBA_LI	يُقال: «رغبة في» وليس «رغبة لـ».
SY_AATIL_AN	الأفصح أن يقال: «عاطل من العمل» أو «متعطل».
SY_TA2MEEN_DHIDDA	الأفصح «تأمين من» أو «مناعة من» بدلاً من «ضد».

D.1.4. Semantics Rules

Rule ID	Description
SE_DECADES_IYAT	يفضل في العقود الكتابة بصيغة «ـينيات» لا «ـينات»، مثل: الثلاثينيات.
SE_MOAKHARAN	يفضل تجنب «مؤخرا» بهذا المعنى، والأفصح: حديثًا أو قريبًا.
SE_MUTAAKID	يفضل استعمال «متحقق» أو «متيقن» بدل «متأكد».

Rule ID	Description
SE_DHATA	يفضل «ذواتا» بدل «ذاتا» في هذا الاستعمال.
SE_KHATIR	يفضل في هذا الاستعمال: شديد الخطر أو محفوف بالخطر بدل «خطير».
SE_KHAMMARA	«الخمارة» بائعة الخمر، ويستحسن للمكان: «مخمرة» أو «حانة».
SE_INDHAHALA	الفعل الأفصح: «ذهل» لا «انذهل».
SE_BIAKMALIHI	يفضل «كله» أو «جميعه» أو «برمته» بدل «بأكمله».
SE_TAHAMMAMA	يفضل استعمال «استحم» بدل «تحمم».
SE_TASHKILU	يستحسن الاستغناء عن «تشكل» في هذا الاستعمال أو استبدالها بـ«هي».
SE_TASAMAMA	الفعل المضعف هنا يدغم، والأفصح: «تصامّ» لا «تصامم».
SE_TATMIN	الأفصح: «طمأنة» لا «تطمين».
SE_TA3KISU	يفضل في هذا السياق «تظهر» أو «تفضح» بدل «تعكس».
SE_JANOOBI	في الظرفية المكانية يفضل «جنوبَ» بدل «جنوبي».
SE_KHESISAN	يفضل «خِصِّيصَى» أو «خاصًا» بدل «خصيصا».
SE_KHALOOQ	يفضل في الوصف هنا: «حسن الخلق» بدل «خلوق».
SE_RAGHMA	يفضل «على الرغم» أو «بالرغم» أو «على» أو «مع» بدل «رغم».
SE_RAFAH	المستعمل هنا «رفات» لا «رفاة».
SE_SHAWYAN	الأفصح في مصدر «شوى»: «شَيًّا» لا «شويا».
SE_ARAYA	يفضل في هذا المعنى «عريانـون» لا «عرايا».
SE_LIWAHDIHI	الأفصح: «وحده» بدل «لوحده».
SE_MAHALAT	جمع «محل» في هذا الاستعمال هو «محالّ» لا «محلات».
SE_NAFOUKH	الأفصح: «يافوخ» بدل «نافوخ».
SE_NASHET	في الوصف يفضل «نشيط» أو «ناشط» بدل «نشط».
SE_WALLATI	يستحسن حذف الواو من «والتي» في هذا الربط.
SE_WALLADHI	يستحسن حذف الواو من «والذي» في هذا الربط.
SE_ITTILA3	الأفصح: «اطّلاع» بدل «إطلاع».
SE_IDHTARADA	الأفصح: «اطرّد» بدل «اضطرد».
SE_MUJBAA	الأفصح: «مجبية» بدل «مجباة».
SE_MOUSOUD	الأفصح: «موصَد» بدل «موصود».
SE_MUASHIRAT	يستحسن في هذا المعنى: إشارات أو علامات أو شواهد أو دلائل بدل «مؤشرات».

Rule ID	Description
SE_MISHWAR	يستحسن تجنب «مشوار» بهذا المعنى، والأفصح: طريق أو مسار أو نزهة.
SE_MACHINE	الأفصح: «مكنة» بدل «ماكينة».
SE_IMKANIYAT	يفضل «إمكانات» بدل «إمكانيات».
SE_TAWAJUD	يفضل في هذا الاستعمال «وجد» بدل «تواجد».
SE_MUSBAQAN	يفضل «مقدمًا» أو «سلفًا» أو «قبلًا» بدل «مسبقا».
SE_WABITTALI	«وبالتالي» تعبير ركيك في هذا السياق، ويستحسن استبداله ببدائل أفصح.
SE_BIMATHABATI	يفضل «بمنزلة» أو «تقوم مقام» بدل «بمثابة».
SE_ISTIBYAN	يفضل «استبانة» بدل «استبيان».
SE_AKHISSAI	يفضل «اختصاصي» بدل «أخصائي».
SE_TOQOS	يفضل «شعائر» بدل «طقوس» في هذا الاستعمال.
SE_BIHASABI	يستحسن تجنب «بحسب» في هذا الاستعمال واستبدالها ببدائل أفصح.
SE_LISALIHIKA	يفضل «لمصلحتك» بدل «لصالحك».
SE_LISALIHI	يفضل «لمصلحة» بدل «لصالح».
SE_I3TABAR	يفضل في هذا الاستعمال «عدّ» بدل «اعتبر».
SE_BURHA	يفضل «هنيهة» بدل «برهة» في هذا السياق.
SE_AAWINA	يفضل «أوان» بدل «آونة».
SE_BAWASIL	في هذا الاستعمال يفضل «بسلاء» أو «باسلون» بدل «بواسل».
SE_MUSTAHTIR	يفضل «مستهين» أو «مستخفّ» بدل «مستهتر» في هذا الاستعمال.
SE_SUWAH	يفضل «سياح» بدل «سواح».
SE_KAKULL	«ككل» ترجمة ركيكة، ويستحسن «كليًا».

D.2. Lexicon Split and Merge Patterns

The lexicon matcher supplements the rule-based system with exact curated patterns. The split and merge entries form an important bridge between pure spelling normalization and grammar-aware error detection.

D.2.1. Split Patterns

Pattern ID	Wrong Surface Form	Correct Form	Purpose
LEX_SPLIT_LAKIN	لا كن	لكن	Common split correction
LEX_SPLIT_HAYTHU_MA	حيث ما	حيثما	Conditional/relative split correction
LEX_SPLIT_KAY_LA	كي لا	كيلا	Common split correction
LEX_SPLIT_BAADAMA	بعد ما	بعدما	Common split correction
LEX_SPLIT_LIMAN	ل من	لمن	Clitic attachment correction
LEX_SPLIT_FI_MA	في ما	فيما	Common split correction
LEX_SPLIT_LIMADHA	ل ماذا	لماذا	Interrogative clitic correction
LEX_SPLIT_INDA_MA	عند ما	عندما	Common split correction
LEX_SPLIT_KULLAMA	كل ما	كلما	Conditional/time expression correction
LEX_SPLIT_RUBBA_MA	رب ما	ربما	Common split correction

D.2.2. Merge Patterns

Pattern ID	Wrong Token	Correct Tokens	Purpose
LEX_MERGE_IN_SHA_ALLAH	انشاءالله	إن شاء الله	Common merge correction
LEX_MERGE_MASHA_ALLAH	ماشاءالله	ما شاء الله	Common merge correction
LEX_MERGE_ALHAMD_LILLAH	الحمدلله	الحمد لله	Common merge correction

Pattern ID	Wrong Token	Correct Tokens	Purpose
LEX_MERGE_SUBHAN_ALLAH	سبحانالله	سبحان الله	Common merge correction
LEX_MERGE_LA_SIYYAMA	لاسيما	لا سيما	Common merge correction
LEX_MERGE_JAZAK_ALLAH	جزاكالله	جزاك الله	Common merge correction
LEX_MERGE_TAWAKKAL_ALLAH	توكلتعلىالله	توكلت على الله	Common merge correction
LEX_MERGE_ASTAGHFIR_ALLAH	استغفرالله	أستغفر الله	Common merge correction
LEX_MERGE_ALLAHU_AKBAR	اللهاكبر	الله أكبر	Common merge correction
LEX_MERGE_BISMILLAH	بسماللهالرحمنالرحيم	بسم الله الرحمن الرحيم	Common merge correction